

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JNANA SANGAMA”, BELAGAVI – 590018



DEEP LEARNING MINI PROJECT REPORT (CS722T2C)

on

“SignalGuard: Deep Learning Based Email Spam and Threat Analysis System”

Submitted by

Rachel Maria Noronha 4SF23CS162

Saanvi Rai 4SF23CS175

Sheza Rizqin Jalaluddheen 4SF23C203

In partial fulfillment of the requirements for the VII semester

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE & ENGINEERING

Under the Guidance of

Ms. Shiji Abraham

Assistant Professor

Department of CSE

at



SAHYADRI

College of Engineering & Management

An Autonomous Institution

MANGALURU

2026 – 27



SAHYADRI
COLLEGE OF ENGINEERING & MANAGEMENT
An Autonomous Institution
MANGALURU

Department of Computer Science & Engineering

DECLARATION

We hereby declare that the entire work embodied in this Mini Project Report titled “SignalGuard: Deep Learning Based Email Spam and Threat Analysis System” has been carried out by us at Sahyadri College of Engineering & Management, Mangaluru, under the supervision of Ms. Shiji Abraham, Assistant Professor, Department of Computer Science & Engineering, in partial fulfillment of the requirements for the VII semester of Bachelor of Engineering in Computer Science & Engineering. We further declare that this report has not been submitted, in part or in full, to this or any other University for the award of any other degree or diploma.

Rachel Maria Noronha (4SF23CS162)

Saanvi Rai (4SF23CS203)

Sheza Rizqin Jalaluddheen (4SF23CS203)

Dept. of CSE, SCEM, Mangaluru



SAHYADRI
COLLEGE OF ENGINEERING & MANAGEMENT
An Autonomous Institution
MANGALURU

Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the Deep Learning Mini Project (CS722T2C) work entitled “SignalGuard: Deep Learning Based Email Spam and Threat Analysis System” has been carried out by Rachel Maria Noronha (4SF23CS162), Saanvi Rai (4SF23CS175) and Sheza Rizqin Jalaluddheen (4SF23CS203), bona fide students of Sahyadri College of Engineering & Management, Mangaluru, in partial fulfillment of the requirements for the VII semester of the Bachelor of Engineering degree in Computer Science & Engineering of the Visvesvaraya Technological University, Belagavi, during the academic year 2026–27. The mini project report has been approved as it satisfies the academic requirements prescribed for the said degree.

Signature of Guide

Ms. Shiji Abraham

Signature of HOD

Dept. of CSE, SCEM

Name of the Examiners

1. _____

2. _____

Signature with date

ACKNOWLEDGEMENT

We take this opportunity to express our sincere gratitude to everyone who supported us throughout the course of this Deep Learning mini project, “SignalGuard: Deep Learning Based Email Spam and Threat Analysis System”

We are deeply thankful to our guide, Ms. Shiji Abraham, Assistant Professor, Department of Computer Science & Engineering, for her valuable guidance, constant encouragement, and constructive feedback at every stage of this project.

We also extend our gratitude to the Head of the Department of Computer Science & Engineering and to the management of Sahyadri College of Engineering & Management, Mangaluru, for providing the infrastructure and academic environment necessary to carry out this work.

Finally, we thank our family and friends for their continuous support and encouragement throughout the duration of this project.

– Project Team 17

ABSTRACT

Unsolicited and malicious e-mail continues to be one of the most common vectors used to defraud individuals and organisations, ranging from bulk advertising spam to more targeted phishing and social engineering attempts. Rule-based and keyword based filters are brittle because attackers can trivially reword or obfuscate their messages, which motivates the use of data driven, deep learning-based text classification. This report presents SignalGuard, a deep-learning e-mail spam classifier that treats spam detection as a binary text-classification problem.

The core of SignalGuard is a sequence model that converts a cleaned e-mail subject and body into a fixed-length sequence of token identifiers, embeds each token in a 96 dimensional space, applies SpatialDropout1D for regularisation. The LSTM output is passed through a ReLU dense layer and a final sigmoid neuron that produces the estimated probability that the e-mail is spam; a threshold of 0.5 is used for the binary decision. The model is trained on the dataset (34,155 e-mails, approximately balanced between spam and non-spam) using a stratified 80/10/10 train-validation test split, the Adam optimiser, binary cross-entropy loss, and class weighting, for 8 epochs with a batch size of 64.

Beyond the core classifier, SignalGuard incorporates several supporting components: A Unicode security analysis stage that flags zero-width characters, bidirectional control characters, mixed script tokens and other adversarial-text patterns; a deterministic, rule based layer that surfaces URL and social-engineering signals that are explicitly kept separate from the neural network's own prediction; a robustness-probing module that measures the stability of the model's decision under harmless textual transformations; and a position-preserving occlusion-based attribution method that estimates the contribution of individual tokens to the predicted spam probability. Untrusted uploaded e-mails are handled through a dedicated set of file-security controls, including extension validation, a 512 KB size limit, binary-content detection and safe parsing, without executing any embedded HTML, fetching remote URLs, or opening attachments.

Keywords: Spam detection, phishing, deep learning, LSTM, natural language processing, probability calibration, adversarial robustness, Unicode security, explainable AI, cybersecurity.

TABLE OF CONTENTS

1. INTRODUCTION.....	1
1.1 Overview	1
1.2 Scope and Motivation	1
2. LITERATURE SURVEY.....	3
2.1 Machine Learning and Deep Learning for E-mail Spam Filtering.....	3
2.2 Phishing Email Detection and Curated Datasets	4
2.3 LSTM-Based Text Classification.....	4
2.4 Adversarial Text Attacks and Unicode Security	5
2.5 Model Calibration.....	5
2.6 Explainable AI and Occlusion-Based Attribution	6
2.7 Summary of the Literature Survey	6
3. PROBLEM FORMULATION	8
3.1 Problem Description	8
3.2 Problem Statement.....	8
3.3 Objectives.....	9
3.4 Functional Requirements	9
3.5 Non-Functional Requirements.....	9
4. PROJECT DESIGN AND IMPLEMENTATION.....	11
4.1 Proposed Project Architecture	11
4.2 High-Level Design.....	12
4.3 Detailed Design.....	14
4.4 Dataset Description.....	14
4.5 Data Preprocessing	15
4.6 Feature Representation	18
4.7 LSTM Model Architecture.....	18

4.8 Training Methodology	21
4.9 Inference Pipeline.....	23
4.10 Calibration.....	25
4.11 Robustness and Security.....	27
4.11.1 Robustness Analysis	27
4.11.2 Unicode Security Analysis	29
4.11.3 URL and Social-Engineering Signals.....	29
4.12 Attribution.....	30
4.13 Web Application.....	32
4.13.1 Architecture	32
4.13.2 API Endpoints	33
4.13.3 Email File Security	34
4.14 Tools and Technologies Used	35
5. RESULTS AND DISCUSSION	37
5.1 Experimental Environment.....	37
5.2 Dataset Results	37
5.3 Model Performance	37
5.4 Confusion Matrix.....	38
5.5 Evaluation Metrics: Formulas and Interpretation	39
Accuracy	39
Precision	40
Recall	40
F1-Score	40
False Positive Rate (FPR).....	40
False Negative Rate (FNR).....	40
5.6 Calibration Results	41
5.7 Robustness Analysis.....	42

5.8 Testing Results	42
5.9 Functional Requirements Results Achieved	43
5.10 Non-Functional Requirements Results Achieved	43
5.11 Comparison with Existing Approaches	44
5.12 Project GUI / System Screenshots	45
5.13 Societal Impact of the Project	46
5.14 SDG Mapping.....	48
6. CONCLUSION AND FUTURE SCOPE	50
6.1 Project Strengths.....	50
6.2 Project Limitations and Future Scope	50
7. REFERENCES.....	53

1. INTRODUCTION

1.1 Overview

Electronic mail remains one of the most widely used channels for personal and organisational communication, and consequently one of the most heavily abused channels for unsolicited advertising (spam), fraud, and phishing. Attackers continuously vary vocabulary, formatting, and even the underlying character encoding of their messages in order to evade static, keyword-based filters, which makes spam detection fundamentally a text-classification problem that benefits from learned, data-driven representations rather than hand-written rules.

SignalGuard is a deep learning e-mail spam classifier that frames this problem as binary text classification: given the subject and body of an e-mail, the system estimates the probability that the message is spam and returns a SPAM or NOT SPAM decision using a threshold of 0.5. The core of the system is a compact Embedding + LSTM neural network. Around this core model, SignalGuard adds a set of supporting components that are common requirements of a trustworthy, deployable text classification system: probability calibration, Unicode level security analysis, a deterministic rule based layer for social-engineering and URL signals, adversarial robustness probing, and a lightweight model output based attribution method for explainability. The complete system is exposed through a web application with, so that the model can be exercised interactively or integrated into other tools.

1.2 Scope and Motivation

Scope

The scope of this project encompasses the design, implementation, and evaluation of a complete deep learning-based email threat analysis system. Specifically, the system focuses on ingesting pasted text or securely uploaded .eml and .txt files up to 512 KB through dedicated file-security controls that prohibit remote URL fetching, script execution, and attachment processing. Within the text processing and machine learning pipeline, it applies consistent normalization followed by a sequence-based architecture utilizing a 96 dimensional embedding layer and a 64 unit Long Short-Term Memory network. Furthermore, the system's scope extends to advanced auxiliary features such as probability calibration via temperature

scaling, deterministic heuristic detection for social-engineering signals, Unicode-security analysis, and an adversarial robustness-probing module. Finally, the project is fully delivered as an interactive Flask-based web application with session history analytics, while intentionally excluding the persistent storage of email bodies to preserve user privacy.

Motivation

SignalGuard addresses three primary limitations common in existing spam and phishing detection systems:

- **Fragility of Rule-Based Filters:** Traditional keyword rules fail against trivial rewording or invisible Unicode obfuscation, necessitating SignalGuard's learned LSTM neural representation rather than static rule sets.
- **Lack of Calibration and Robustness:** Many systems report raw accuracy without probability calibration or decision stability testing; SignalGuard explicitly incorporates temperature scaling, adversarial probing, and transparent uncertainty handling.
- **Missing Application Security:** Academic classifiers are rarely packaged into secure, end-to-end web applications that safely handle untrusted user inputs.

2. LITERATURE SURVEY

This chapter surveys prior academic work relevant to the different components of SignalGuard: classical and deep learning approaches to e-mail spam and phishing classification, the datasets commonly used to evaluate such systems, adversarial and Unicode based text attacks, model calibration, and model output based explainability. Each source is cited using the author year convention and is listed in full in the References chapter. Where a claim could not be independently verified at the time of writing, it has been noted as such rather than presented as an established fact.

2.1 Machine Learning and Deep Learning for E-mail Spam Filtering

Dada et al. (2019) provide a widely cited systematic review of machine-learning approaches to e-mail spam filtering, covering classical algorithms such as Naïve Bayes, Support Vector Machines, and Artificial Neural Networks, and discussing evaluation measures such as the Total Cost Ratio that weigh false positives and false negatives asymmetrically – a consideration that remains relevant to SignalGuard's own error-rate reporting in Chapter 5. The review also catalogues the publicly available spam corpora that have historically been used for benchmarking, several of which (Enron, SpamAssassin, Ling, Nazario, Nigerian Fraud/CLAIR) are also present among the datasets available to the SignalGuard project, even though the currently shipped model is trained only on CEAS_08.

More recent work has moved from hand-engineered, TF-IDF-style features toward sequence models capable of learning their own representations directly from raw token sequences. Comparative studies of Recurrent Neural Network variants for spam and phishing text classification have repeatedly found that LSTM-based architectures outperform simpler feed-forward or convolutional baselines on this task, which is consistent with the architectural choice made for SignalGuard's core classifier (an Embedding layer followed by a single LSTM layer and a dense sigmoid head). Reported accuracy figures in this line of work are generally very high (often above 98–99% on a single held-out split of a single dataset), which mirrors the test-set performance obtained for SignalGuard on CEAS_08 and motivates the cross-dataset caveats discussed later in this section.

2.2 Phishing Email Detection and Curated Datasets

Champa, Rabbi and Zibran (2024) curated and released seven phishing-and-legitimate e-mail datasets comprising more than 200,000 e-mail instances, and used them to evaluate the sensitivity of several machine-learning algorithms to different feature choices; the same authors' companion study (Rabbi, Champa and Zibran, 2023) applied NLP and ML techniques directly to phishing-e-mail text and reported high accuracy, precision and recall using a similar subject/body feature representation to the one used by SignalGuard. Related work by the same research group (Champa, Rabbi and Zibran, 2024b) specifically investigated why phishing e-mails escape detection, reporting that detectors trained on one dataset generalise poorly to previously unseen or more recent phishing e-mails – a finding of direct relevance to SignalGuard, whose model is trained and evaluated only on the single-source CEAS_08 corpus.

This cross-dataset generalisation concern has been substantiated more broadly by Pajola et al. (2025), who reassessed a range of feature-based and transformer-based phishing detectors (including TF-IDF classifiers and a fine-tuned DistilBERT model) across eight widely used benchmark datasets, including CEAS. Their results show that models trained and tested on the same dataset consistently achieve near-perfect performance (F1-scores frequently above 0.98, matching the order of magnitude of SignalGuard's own CEAS_08 test-set metrics reported in Chapter 5), but that the same models suffer a substantial drop in F1-score – sometimes to near zero – when evaluated on a different dataset drawn from a different time period or e-mail source. The authors explicitly caution that near-perfect same-dataset accuracy should not be read as evidence of real-world effectiveness. Al-Subaiey et al. (2024) similarly describe a web-deployed phishing-detection platform trained on a large public dataset that reports an F1-score of 0.99 while explicitly integrating explainable-AI methods to increase user trust in the system's decisions, a design goal shared by SignalGuard's attribution component. These findings collectively motivate the explicit “domain-shift” limitation discussed for SignalGuard in Chapter 5 and Chapter 6: a high score on CEAS_08 is evidence of the model's ability to fit that particular data distribution, not a general guarantee against modern or previously unseen phishing campaigns.

2.3 LSTM-Based Text Classification

The Long Short-Term Memory (LSTM) network, originally introduced to overcome the vanishing-gradient problem of plain recurrent networks, has become one of the standard

sequence-modelling building blocks for text classification tasks where word order and long-range dependency information are informative, such as detecting sentence-level manipulation cues in spam and phishing text. In the specific context of e-mail-style text classification, published comparisons of LSTM, Gated Recurrent Unit (GRU), and hybrid LSTM–GRU or CNN–LSTM architectures generally report that LSTM-family models achieve strong accuracy on spam and phishing corpora, while also being noted as more computationally demanding than purely feed-forward alternatives. This body of work supports the choice of a single-layer, 64-unit LSTM for SignalGuard, which is intended to keep the model small enough for interactive, single-request inference in a web application while still capturing the sequential structure of e-mail text.

2.4 Adversarial Text Attacks and Unicode Security

SignalGuard's Unicode-security-analysis component is motivated by a distinct line of security research on how the internal, machine-readable representation of text can be manipulated without changing its visible appearance to a human reader. Boucher, Shumailov, Anderson and Papernot (2022) demonstrated a broad class of “imperceptible” adversarial attacks against NLP systems using invisible characters, homoglyph substitutions, character reordering, and deletions, showing that such perturbations can substantially change a downstream model's output while remaining visually indistinguishable to a human. Boucher and Anderson (2023) later showed, in the “Trojan Source” work, that the Unicode Bidirectional Algorithm's directional-override control characters can be used to make text that is logically encoded in one order display in a different, misleading order – a technique with an evident analogue in e-mail-based social engineering, where the visible and machine-processed content of a message could in principle be made to diverge. These findings directly motivate SignalGuard's dedicated detection of zero-width characters, bidirectional-control characters, other format/control characters, unusual spaces, and mixed-script or homoglyph patterns as a security-analysis layer that is kept separate from, and reported alongside, the LSTM's own prediction.

2.5 Model Calibration

A model's raw sigmoid or softmax output is not guaranteed to behave as a well-calibrated probability, meaning that a predicted confidence of, say, 90% need not correspond to the model being correct 90% of the time. Desai and Durrett (2020) studied the calibration of pre-trained Transformer language models on several natural-language-understanding tasks and found that,

although such models are reasonably well calibrated in-domain, temperature scaling – a simple post-hoc rescaling of the pre-sigmoid logit by a single learned scalar temperature – further and reliably reduces calibration error without changing the model's ranking of predictions (and therefore without changing its accuracy). SignalGuard adopts the same temperature-scaling technique, fitting a single temperature parameter on the validation split and reporting the resulting reduction in Expected Calibration Error and Negative Log-Likelihood in Chapter 5, consistent with the methodology described in this line of calibration research.

2.6 Explainable AI and Occlusion-Based Attribution

Interpretability methods for neural text classifiers broadly fall into gradient-based approaches (such as Integrated Gradients), surrogate-model approaches (such as LIME or SHAP), and perturbation- or occlusion-based approaches that directly observe how a model's output changes when parts of the input are removed or replaced. Li, Monroe and Jurafsky (2017) proposed a general representation-erasure methodology for interpreting neural NLP models, in which individual words, phrases, or hidden units are erased and the resulting change in the model's output is used to estimate the erased element's contribution to the original decision. SignalGuard's token-attribution component follows this same occlusion principle at the input-token level: an active token is replaced with the tokenizer's out-of-vocabulary token while keeping the sequence length and every other token's position unchanged, and the resulting change in predicted spam probability is reported as that token's attribution score. Because this method only ever queries the trained model's output and does not inspect gradients or hidden activations, it is explicitly distinct from gradient-based methods such as Integrated Gradients and from surrogate-model methods such as SHAP, and this report is careful throughout to describe it only as a model-output-based occlusion attribution rather than by either of those other names.

2.7 Summary of the Literature Survey

Taken together, the surveyed literature supports three of the central design decisions behind SignalGuard: (i) an Embedding + LSTM deep-learning classifier is a well-established and effective architecture for spam and phishing text classification; (ii) reported near-perfect accuracy on a single benchmark dataset such as CEAS_08 is a well-documented phenomenon in this literature and is not, by itself, evidence of robustness to modern or previously unseen e-mail traffic, which motivates the explicit domain-shift caveats maintained throughout this

report; and (iii) calibration, adversarial/Unicode robustness, and lightweight model-output-based explainability are all independently studied concerns in the wider NLP and security literature, each of which SignalGuard incorporates as a distinct, clearly labelled component rather than conflating them with the core neural prediction.

3. PROBLEM FORMULATION

3.1 Problem Description

Spam and phishing e-mails cause direct financial loss, credential theft, and malware delivery, and their content is continually rephrased by attackers to evade static filters. A practical detector must therefore learn from the statistical and sequential structure of e-mail text itself rather than relying purely on fixed keyword lists. At the same time, an e-mail-analysis system is itself a piece of software that receives untrusted input (arbitrary text and, optionally, uploaded files), so it must be designed defensively: it must not execute any content embedded in an e-mail, must bound the resources it consumes while parsing an uploaded file, and must not silently trust the visible text of a message when that text may have been manipulated using invisible or reordered Unicode characters.

SignalGuard addresses this by combining a supervised deep-learning classifier, trained on labelled e-mail text, with a small number of clearly separated supporting layers - calibration, Unicode security analysis, deterministic social engineering/URL signal extraction, robustness probing, and token attribution that make the system's output more trustworthy and easier to inspect without altering the core classification decision produced by the neural network.

3.2 Problem Statement

Given the subject and body text of an e-mail (submitted directly or extracted from an uploaded .eml or .txt file), design and implement a deep-learning system that: (i) preprocesses and normalises the text in a manner robust to common obfuscation techniques; (ii) classifies the e-mail as SPAM or NOT SPAM using a trained Embedding + LSTM neural network, together with a calibrated probability and a discrete risk band; (iii) surfaces Unicode level anomalies and rule based social engineering/URL indicators as supporting evidence, explicitly distinguished from the neural network's own prediction; (iv) reports the stability of its decision under harmless textual transformations; (v) explains its prediction at the level of individual input tokens using a model output-based attribution method; and (vi) exposes all of the above through a secure, web accessible interface and JSON API.

3.3 Objectives

The specific objectives of the SignalGuard project are as follows:

- **Model Training & Classification:** To design and train an Embedding and LSTM deep learning binary classifier and estimate email spam probabilities.
- **Advanced Analysis & Preprocessing:** To implement a rigorous text preprocessing pipeline, a Unicode security analysis module.
- **Secure Web Application Delivery:** To deploy the system via a Flask-based web application with secure file-handling controls for .eml and .txt files, automated test validation, and transparent limitations reporting.

3.4 Functional Requirements

Table 3.1 lists the functional requirements (FR) implemented by SignalGuard.

ID	Functional Requirement
FR-1	The system shall accept an e-mail subject and body as direct text input.
FR-2	The system shall accept an uploaded .eml or .txt file as an alternative input, up to 512 KB.
FR-3	The system shall classify the e-mail as SPAM or NOT SPAM using the trained LSTM model.
FR-4	The system shall report Unicodesecurity findings for the submitted text.
FR-5	The system shall, on request, run robustness probes and report a stability score.
FR-6	The system shall, on request, compute token level attribution scores within a specified budget.
FR-7	The system shall maintain and expose a history of past analyses, with an option to clear it.
FR-8	The system shall expose all of the above functionality through a documented JSON API.

Table 3.1: Functional requirements of SignalGuard

3.5 Non-Functional Requirements

Table 3.2 lists the non-functional requirements (NFR) considered in the design of SignalGuard. Where a requirement has not been formally load-tested, this is stated explicitly rather than assumed.

ID	Category	Requirement
NFR-1	Performance	A single analysis request should complete within a short, interactive latency suitable for a web request; the API response includes a timing breakdown for transparency. No formal load/throughput testing has been performed.
NFR-2	Security	Uploaded files must be validated by extension and size, scanned for binary content, and parsed without executing HTML, fetching URLs, running macros, or opening attachments.
NFR-3	Reliability	The trained model, tokenizer and metadata are loaded once at start-up and reused for every request; inference must be safe to call concurrently from multiple requests (thread-safe model access).
NFR-4	Usability	The web interface should present the prediction, probability, risk band, and supporting analyses in a clear and organised layout.
NFR-5	Maintainability	Preprocessing, modelling, calibration, security analysis, robustness analysis, and attribution should be implemented as separable modules.
NFR-6	Robustness	The system should measure and report, rather than silently assume, the stability of its own predictions under harmless input transformations.

Table 3.2: Non-functional requirements of SignalGuard

4. PROJECT DESIGN AND IMPLEMENTATION

4.1 Proposed Project Architecture

Figure 4.1 shows the end-to-end architecture of SignalGuard. The left-hand path (blue and orange boxes) is the primary classification pipeline: a user submits an e-mail through the web interface or uploads a .eml/.txt file; the input passes through security validation, text extraction; the cleaned text is tokenised, padded, embedded, and passed through the LSTM classifier; the sigmoid output is thresholded at 0.5 to produce the SPAM / NOT SPAM decision, alongside a calibrated confidence and a discrete risk band. The right-hand branches are supporting analyses Unicode security analysis, URL/social-engineering signal extraction, robustness probing, token attribution, and probability calibration that are computed alongside the main classification path and reported separately in the API response, rather than being merged into the neural network's own decision.

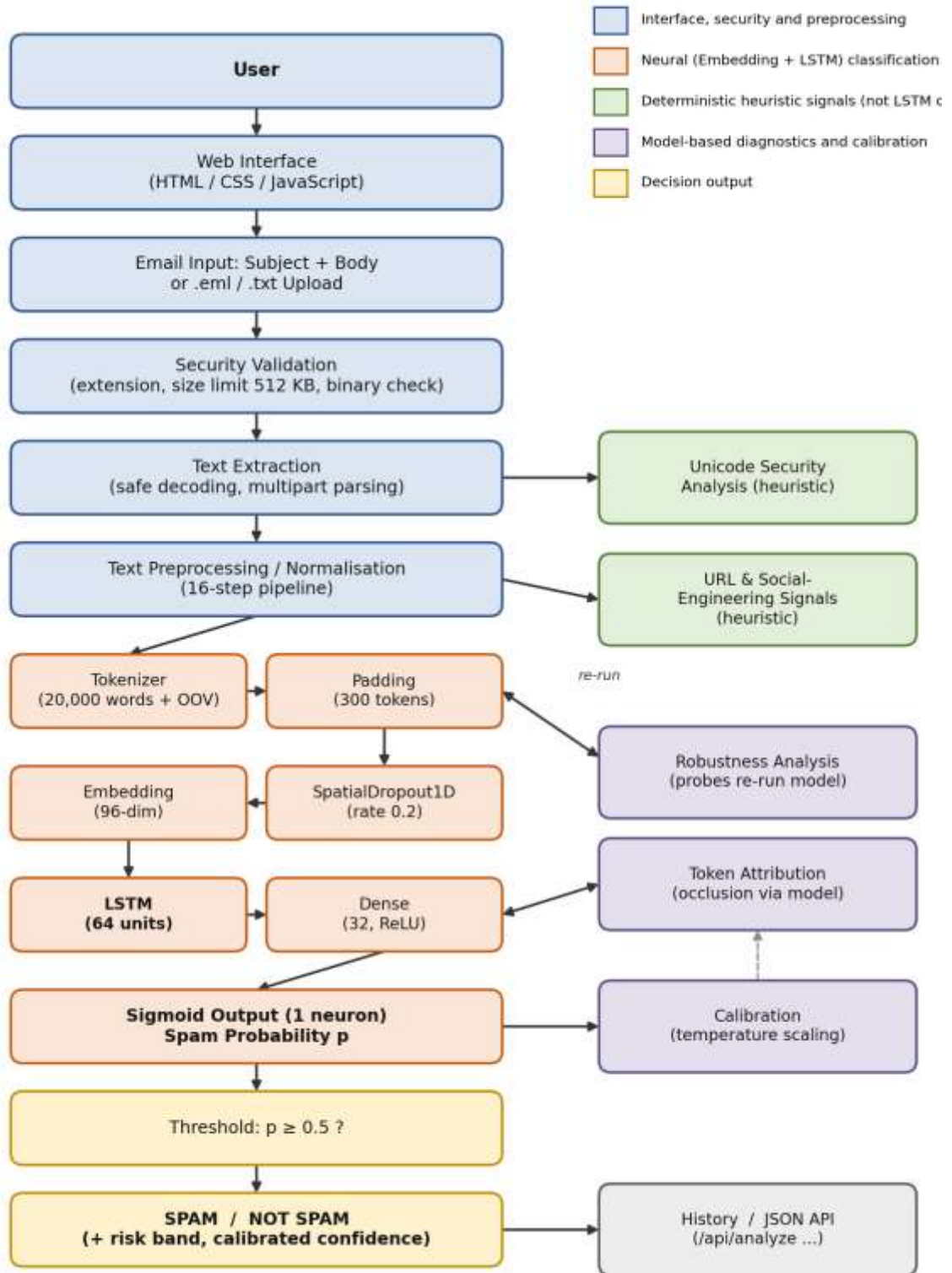


Figure 4.1: Proposed Architecture of SignalGuard

4.2 High-Level Design

At a high level, SignalGuard can be described as a data-flow between seven processes, shown in Figure 4.2 using standard data-flow diagram notation: input validation (1.0), text extraction

and preprocessing (2.0), tokenisation and LSTM inference (3.0), calibration/thresholding/risk assignment (4.0), Unicode and URL/social-engineering signal extraction (5.0), robustness probing and token attribution (6.0), and response composition together with history logging (7.0). The model artifacts (trained model weights, tokenizer, metadata and calibration parameters) are treated as a shared data store (D1) accessed by both the primary inference process and the robustness/attribution processes, since both re-use the same trained model rather than maintaining separate copies.

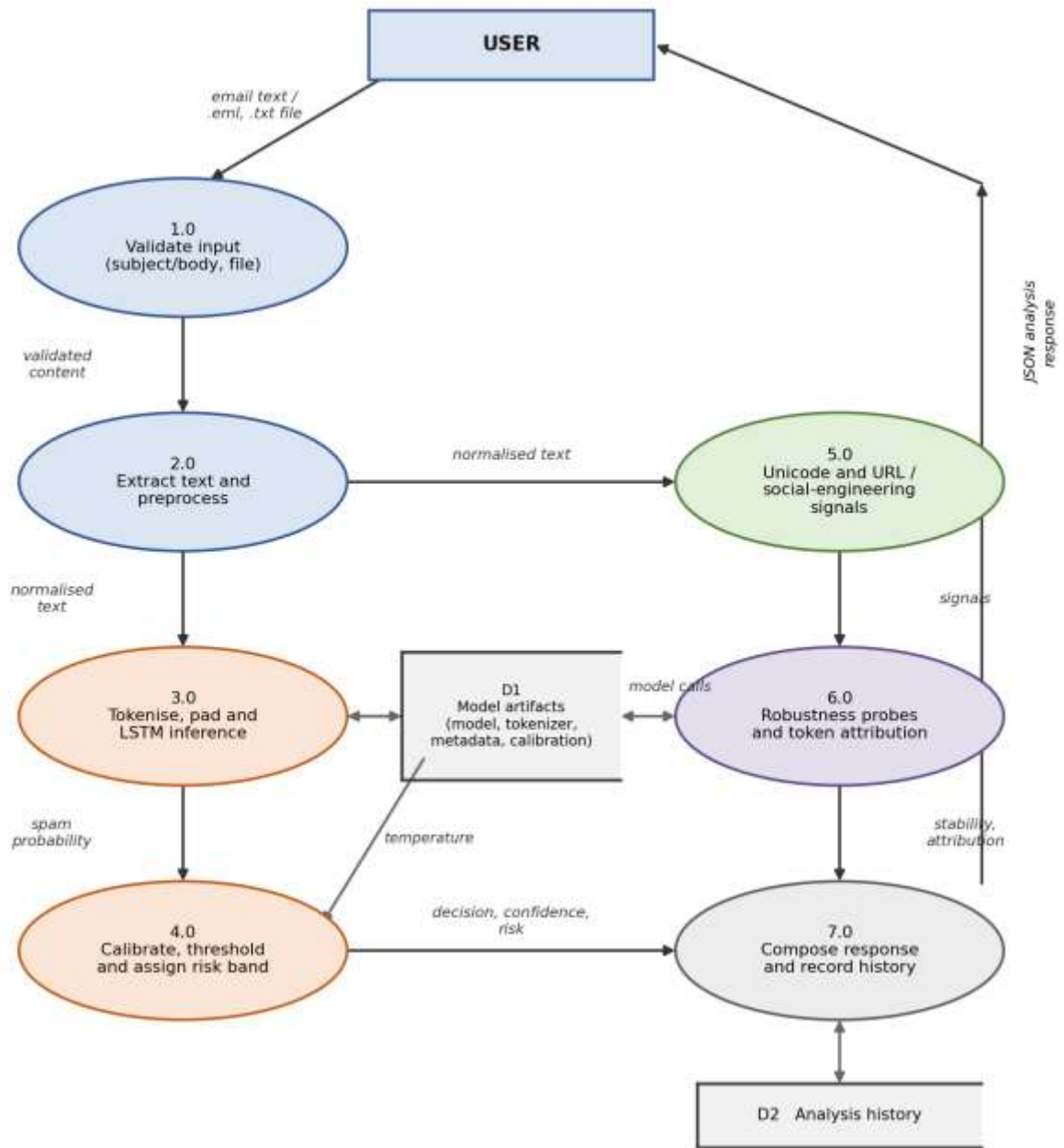


Figure 4.2: Data-Flow Diagram of SignalGuard

4.3 Detailed Design

The following sub-sections describe each component of SignalGuard in detail: the dataset used for training (4.4), the text-preprocessing pipeline (4.5), the feature/sequence representation fed to the network (4.6), the LSTM model architecture (4.7), the training methodology (4.8), the inference pipeline (4.9), probability calibration (4.10), robustness and security analysis (4.11), token attribution (4.12), the web application (4.13), and the tools and technologies used for implementation (4.14).

4.4 Dataset Description

The SignalGuard project has access to several public e-mail datasets for potential use – CEAS_08.csv, email_data.csv, email_dataset_classification.csv, Enron.csv, Ling.csv, Nazario.csv, Nigerian_Fraud.csv, phishing_email.csv, and SpamAssasin.csv but the model that is currently trained and shipped with the application uses only CEAS_08.csv as its training source; it is not trained on a combination of all nine datasets, and this distinction is maintained consistently throughout this report.

The dataset used (CEAS_08) contains 34,155 labelled e-mails, of which 17,240 (50.48%) are labelled NOT SPAM and 16,915 (49.52%) are labelled SPAM, making the dataset approximately balanced between the two classes, as shown in Figure 4.3(a). This near balance means that accuracy is a reasonably informative metric for this dataset, although precision, recall and the confusion matrix are still reported in full, since class balance alone does not guarantee that false positives and false negatives are equally costly in a deployed spam filter.

The dataset is split into training, validation and test partitions using a stratified 80/10/10 split, so that the proportion of spam and non-spam e-mails is preserved across all three partitions. The resulting partition sizes are 27,323 e-mails for training, 3,415 for validation, and 3,416 for testing, as shown in Figure 4.3(b) and summarised in Table 4.1. Stratification is used because, without it, a random split could by chance produce a validation or test partition with a meaningfully different class balance than the training partition, which would make the validation based calibration (Section 4.10) and the test-set evaluation (Chapter 5) less representative of the model's true, balanced class behaviour.

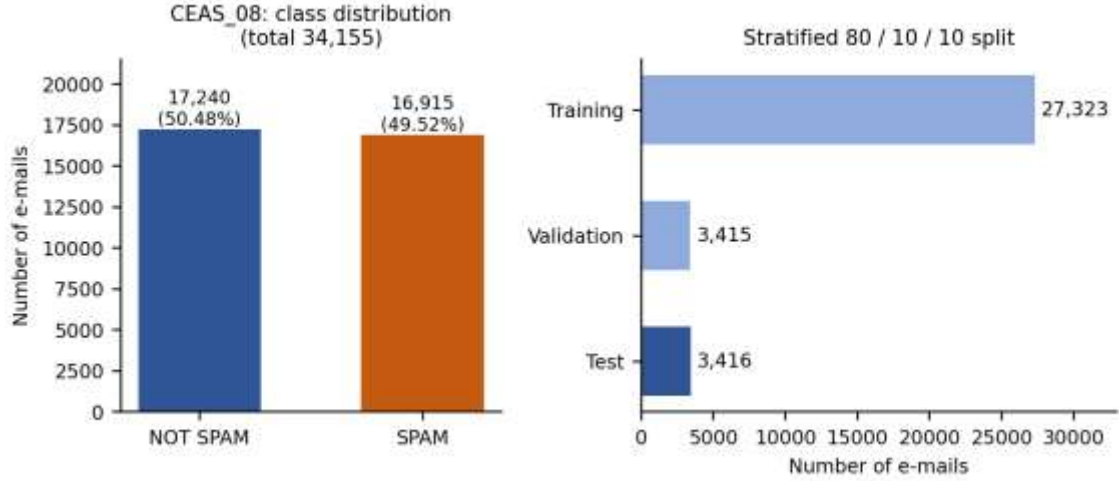


Figure 4.3: CEAS_08 class distribution (a) and stratified data split (b)

Split	Number of e-mails	Percentage
Training	27,323	80.00%
Validation	3,415	10.00%
Test	3,416	10.00%
Total	34,155	100.00%

Table 4.1: CEAS_08 stratified train/validation/test split

Dataset preparation includes label encoding (mapping the dataset's textual or numeric spam/ham labels to a consistent $\{0, 1\}$ representation while explicitly supporting the label formats present in the source files), removal of duplicate e-mails to avoid the same message appearing in more than one split (which would otherwise leak information from the training set into the validation or test set), and the null-handling and length-limiting steps described as part of preprocessing in Section 4.5. Because the CEAS_08 corpus does not include an explicit 'Sender', 'Receiver' or 'Date' field used by the model, only the Subject and Body text fields are used as model input, consistent with the architecture described in Section 4.7.

4.5 Data Preprocessing

Because spam, phishing and other adversarial e-mail content is deliberately written to evade automated filters, text preprocessing in SignalGuard is designed to normalise not only ordinary linguistic variation but also common obfuscation techniques hidden HTML markup, disguised links, invisible or reordered Unicode characters, and excessive character repetition used to

defeat simple keyword matching. Every subject and body string passes through the following sixteen-step pipeline, shown in Figure 4.4, before it is tokenised:

1. Convert null/missing values to empty strings, so that a missing subject or body does not break downstream processing.
2. Decode HTML entities (for example, converting “&” back to “&”), so that entity-encoded text is not treated as literal markup.
3. Apply Unicode NFKC normalisation, which maps visually or semantically equivalent Unicode representations of a character onto a single canonical form.
4. Remove invisible Unicode characters that have no visible glyph but are still processed as tokens.
5. Remove zero-width spaces, which are frequently inserted inside words to break up keyword matches while remaining invisible to a human reader.
6. Remove word-joiner characters, used similarly to zero-width spaces to alter tokenisation without a visible effect.
7. Remove bidirectional-control marks, which can otherwise reorder how text is displayed relative to how it is encoded.
8. Remove other unsupported Unicode control and format characters that serve no legitimate display purpose in e-mail text.
9. Parse and strip HTML markup (including hidden or off-screen HTML elements), so that the model sees plain text rather than markup tags.
10. Mask URLs with the placeholder token URLTOKEN, so that the model can learn that “a URL is present” as a generalisable signal without over-fitting to specific domains seen only in training.
11. Mask e-mail addresses with the placeholder token EMAILTOKEN, for the same reason.
12. Mask phone numbers with the placeholder token PHONETOKEN.
13. Reduce excessive repeated characters (for example, collapsing “FREE!!!!!!”-style repetition), which is a common spam-formatting pattern that would otherwise inflate the vocabulary with near-duplicate tokens.
14. Normalise whitespace (collapsing multiple spaces, tabs and newlines).
15. Limit the subject to a maximum of 2,000 characters.
16. Limit the body to a maximum of 80,000 characters, bounding the computational cost of preprocessing pathological inputs.

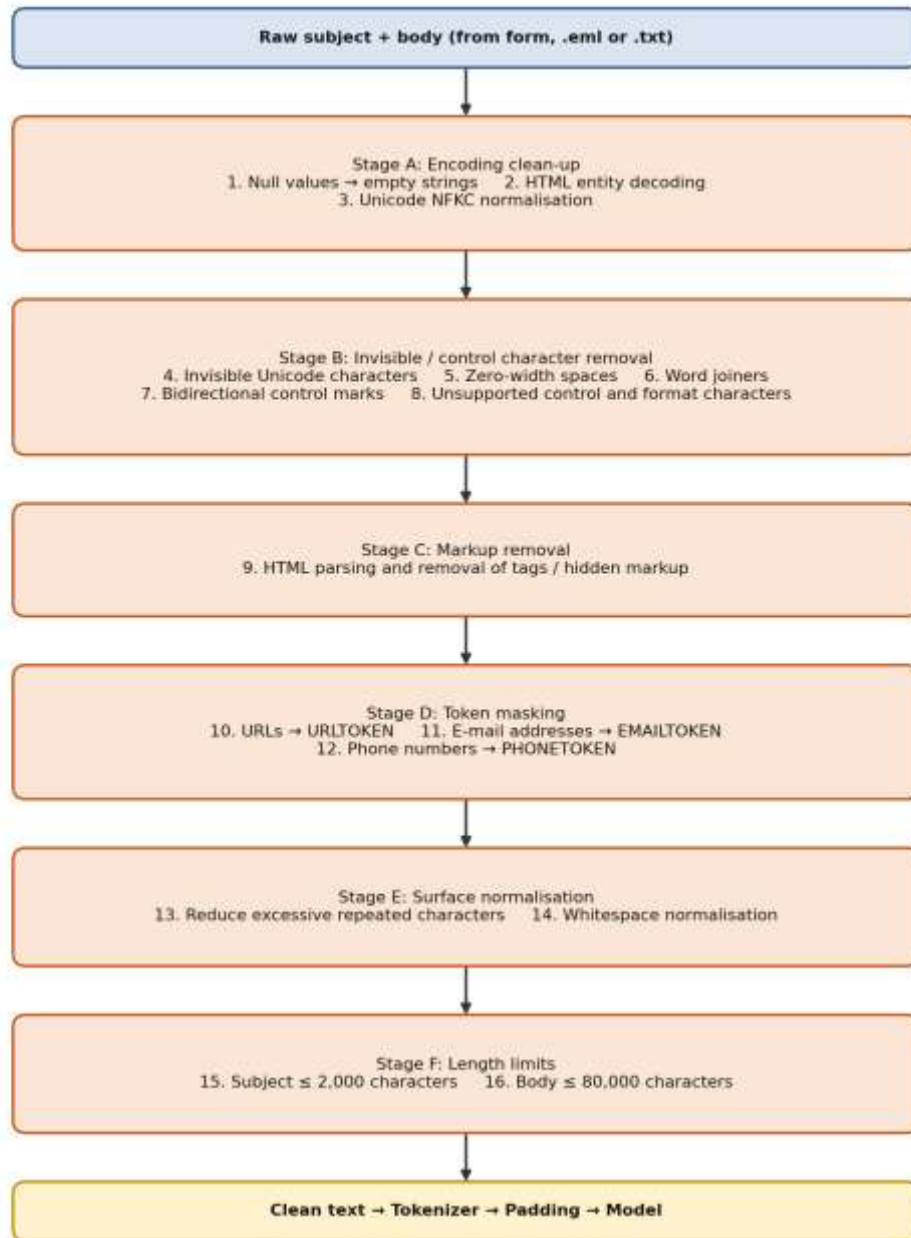


Figure 4.4: Data Preprocessing Pipeline

This ordering matters: HTML parsing (step 9) is performed after the encoding and invisible-character clean-up (steps 1–8) so that an attacker cannot use invisible or control characters to disguise HTML tags from the parser, and masking (steps 10–12) is performed after HTML removal so that URLs and addresses hidden inside markup attributes are still normalised. Preprocessing is deliberately kept separate from feature representation (Section 4.6): its output is always clean, plain, length-bounded text, regardless of which downstream representation (sequence-based, for SignalGuard) ultimately consumes it.

4.6 Feature Representation

SignalGuard does not use TF-IDF or any other bag-of-words feature representation as its primary input to the classifier. Instead, cleaned text is converted into a sequence representation suitable for a recurrent neural network:

Tokenizer → Integer token IDs → Padding → Embedding vectors

A Keras-style Tokenizer is fitted with a vocabulary size of 20,000 words plus a reserved out-of-vocabulary token, <OOV>, that absorbs any word not among the 20,000 most frequent training-set words. Crucially, the tokenizer is fitted only on the training split (never on the validation or test split), so that the vocabulary itself does not leak any information from held-out data; a word that first appears only in the validation or test set is treated as <OOV>, exactly as an unseen word would be treated at real-world inference time. Each e-mail's cleaned text is converted into a sequence of integer token identifiers, then padded or truncated to a fixed length of 300 tokens; both padding and truncation are applied at the end of the sequence (post-padding, post-truncation), so that the beginning of the e-mail – subject and opening lines, which typically carry a disproportionate share of spam/phishing signal – is preserved rather than trimmed away.

This fixed-length integer sequence is passed to a trainable Embedding layer (Section 4.7), which maps each token identifier to a dense, 96-dimensional vector. A sequence representation of this kind is well suited to an LSTM-based classifier because it preserves word order and adjacency information (unlike a bag-of-words/TF-IDF representation, which discards ordering entirely), allowing the LSTM to learn sequential patterns such as an urgency phrase immediately preceding a masked URLTOKEN, which would be indistinguishable from the same two elements appearing far apart in a bag-of-words representation.

4.7 LSTM Model Architecture

Table 4.2 summarises the layer-by-layer architecture of the SignalGuard classification model, and Figure 4.5 shows the same architecture as a labelled block diagram together with an unrolled view of the LSTM layer's internal recurrence.

Layer	Configuration	Purpose
Input	Sequence length 300 (integer token IDs)	Fixed-length numerical representation of the pre-processed e-mail text.
Embedding	Vocabulary 20,000; output dimension 96	Learns a dense vector representation for every token, so semantically or functionally similar tokens (e.g. different but similarly-used spam trigger words) end up with similar vectors.
SpatialDropout1D	Rate = 0.2	Regularisation that drops entire embedding-feature channels (rather than individual, scattered elements) during training, which is better suited to sequence embeddings than standard dropout and helps prevent over-fitting to specific tokens.
LSTM	64 units	Processes the embedded sequence one token at a time, maintaining a hidden state and a cell state that let the network retain information over long distances in the e-mail text; the final hidden state summarises the whole sequence.
Dense	32 neurons, ReLU activation	Projects the 64-dimensional LSTM summary into a further non-linear representation, giving the network additional capacity to combine sequence features before the final decision.
Dense (Output)	1 neuron, Sigmoid activation	Squashes the final scalar into the (0, 1) range, interpreted as the estimated probability that the e-mail is spam.

Table 4.2: LSTM model architecture summary

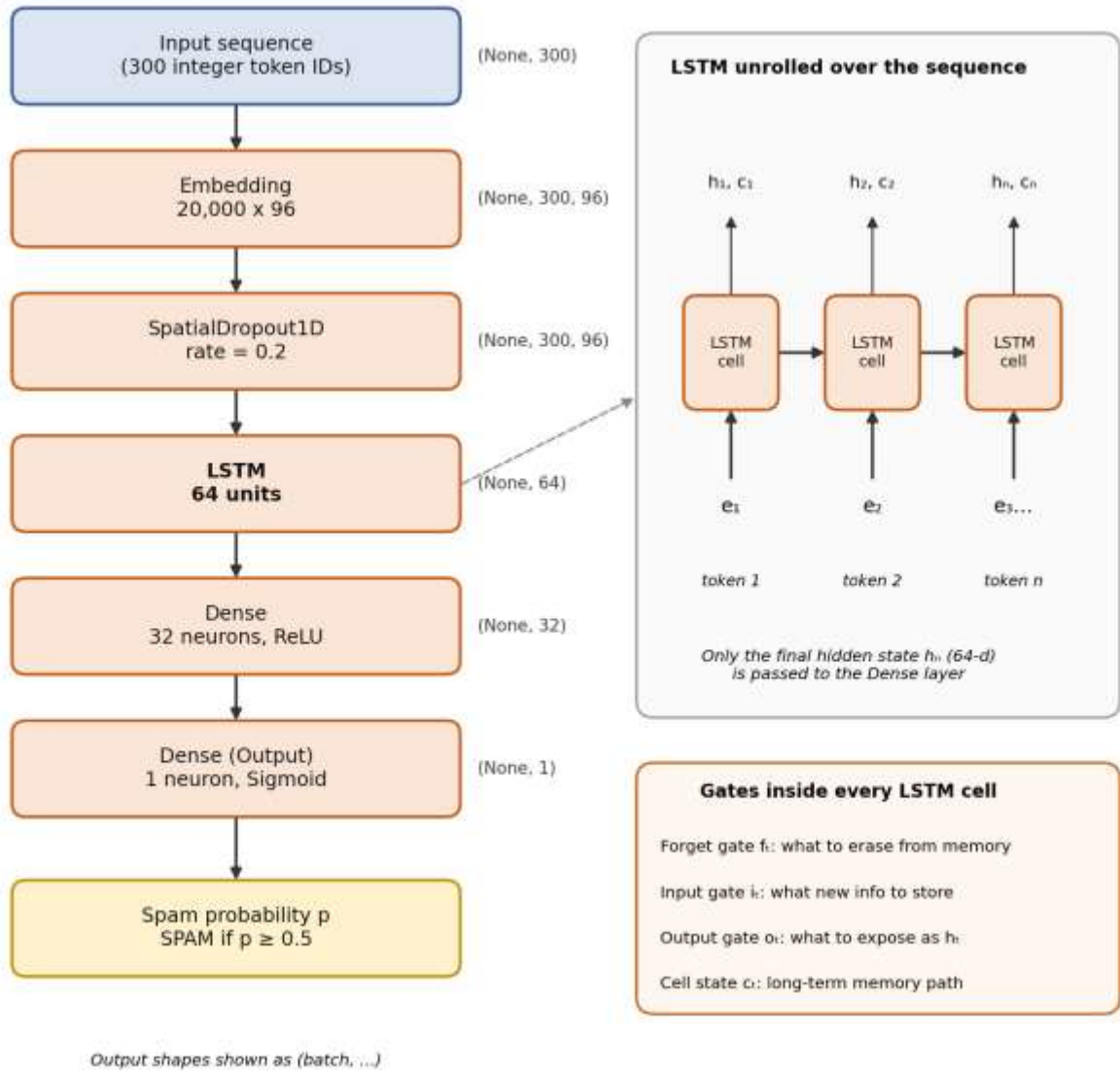


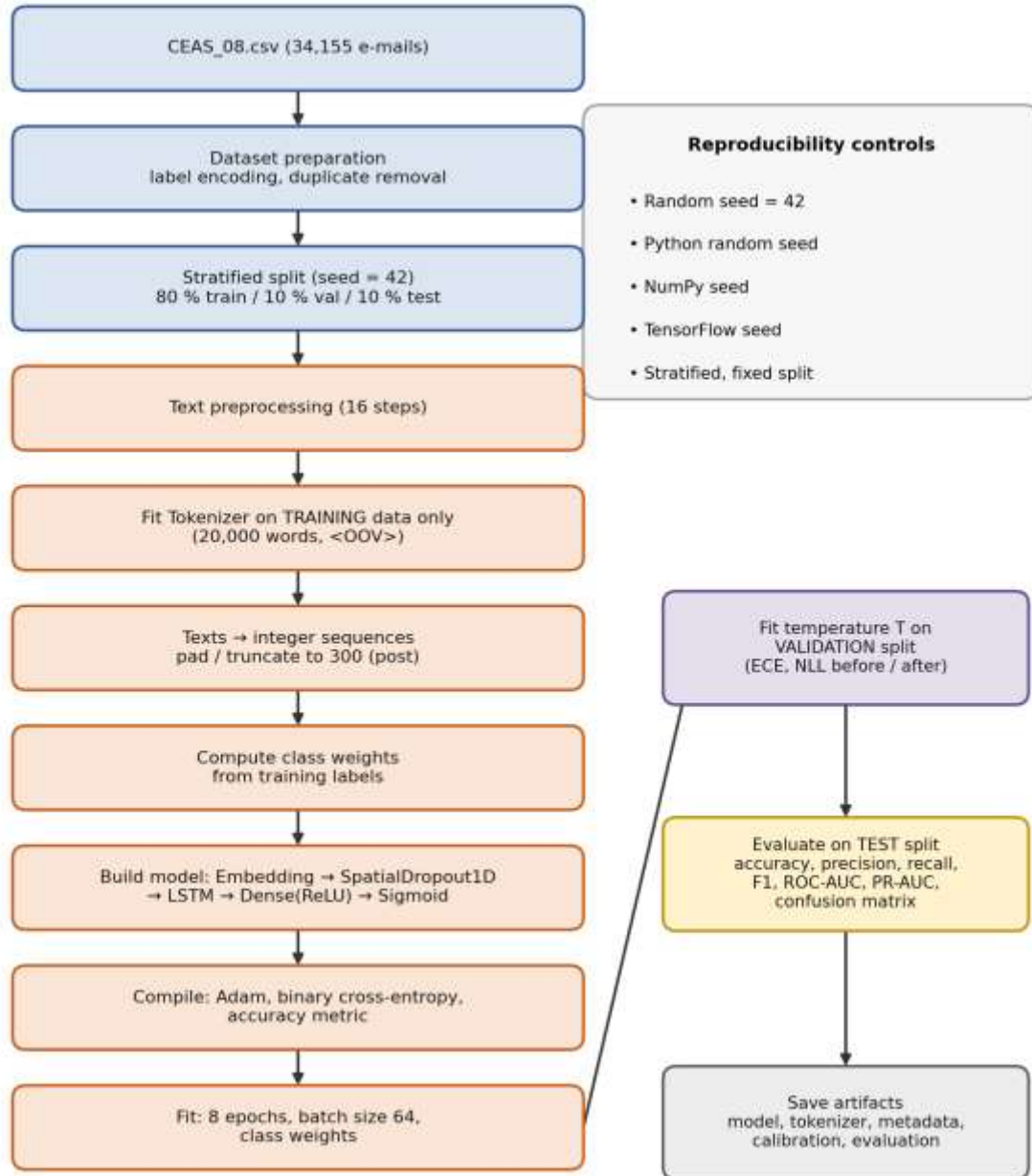
Figure 4.5: LSTM Architecture used by SignalGuard

Each LSTM cell contains three gates, a forget gate that decides what information to discard from the cell state, an input gate that decides what new information to add, and an output gate that decides what part of the (possibly long-term) cell state to expose as the hidden state at that step which together let the network retain relevant context (such as an early urgency phrase) across many intervening tokens without the vanishing-gradient problems that affect plain recurrent networks. Only the final hidden state, a 64-dimensional summary of the entire 300-token sequence, is passed forward to the Dense(32, ReLU) layer and then to the final Dense(1, Sigmoid) output neuron, which produces the spam probability p used for classification: the email is classified as SPAM if $p \geq 0.5$, and as NOT SPAM otherwise.

The model is compiled with the Adam optimiser and the binary cross-entropy loss function, and accuracy is tracked as an additional training metric. Binary cross-entropy is the standard loss function for a single sigmoid output binary classifier because it directly penalises the model's predicted probability being far from the true 0/1 label, rather than only penalising the final thresholded decision; Adam is used because it adapts the learning rate per parameter and is a robust default optimiser for recurrent architectures such as this one.

4.8 Training Methodology

Figure 4.6 shows the complete training pipeline used to produce the SignalGuard model artifacts. Training begins from the CEAS_08 dataset, applies dataset preparation (label encoding and duplicate removal), performs the stratified 80/10/10 split described in Section 4.4, and then applies the same 16-step text-preprocessing pipeline described in Section 4.5 before fitting the Tokenizer on the training split only and converting every split to padded, 300-token integer sequences.



No early stopping and no L2 regularisation in the current implementation

Figure 4.6: Training Pipeline

Class weights are computed from the training-split label distribution and passed to the training loop so that the (slightly) less frequent class is not systematically under emphasised by the loss function. The model is compiled with the Adam optimiser, binary cross-entropy loss, and accuracy as a monitored metric, and is trained for 8 epochs with a batch size of 64. Reproducibility is addressed by fixing a single random seed (42) and applying it consistently to Python's own random module, NumPy, and TensorFlow, in addition to using the same fixed, stratified data split for every run.

It should be stated plainly that the current implementation does not use early stopping (training always runs for the fixed 8 epochs rather than stopping automatically when validation performance plateaus) and does not use L2 (weight-decay) regularisation; the only regularisation mechanism present in the model is the SpatialDropout1D layer described in Section 4.7. This is noted here as a design characteristic of the current implementation, and is revisited as a limitation and a future-work item in Chapter 5 and Chapter 6 respectively.

After training, the model is evaluated once on the held-out test split (never used for any training or hyper-parameter decision) to obtain the metrics reported in Chapter 5, and a separate temperature-scaling calibration step (Section 4.10) is fitted using the validation split. Finally, the trained model, the fitted tokenizer, run metadata, the calibration parameters, and the offline evaluation metrics are all persisted as artifacts that are loaded once by the web application at start-up (Section 4.9).

4.9 Inference Pipeline

Figure 4.7 distinguishes the two phases of SignalGuard's inference pipeline. In Phase 1, performed once when the Flask application starts, the trained model, the fitted tokenizer, run metadata, and the calibration parameters are loaded into memory, and a single warm-up inference call is made so that any one-time initialisation cost inside the deep-learning framework (such as graph construction or kernel selection) is paid before the first real user request arrives, rather than being visible as extra latency on that first request.

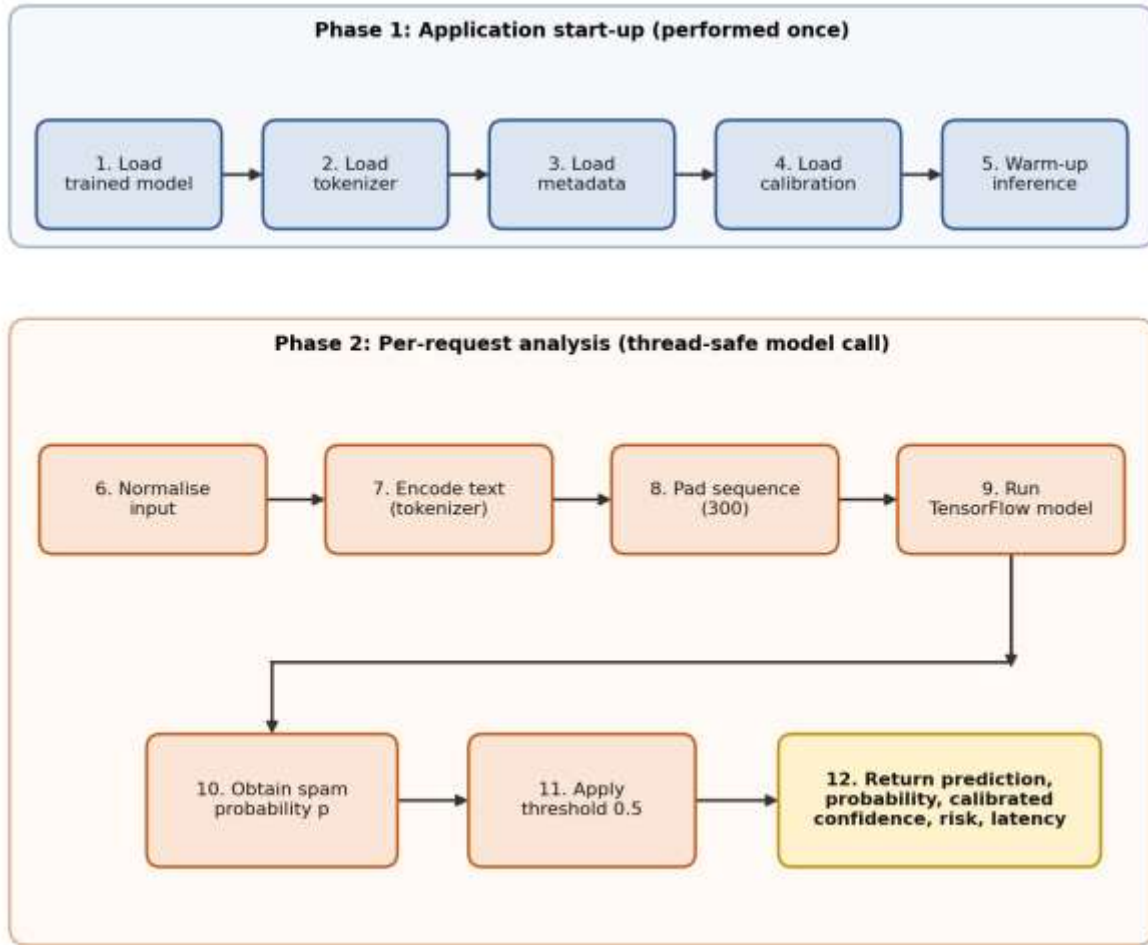


Figure 4.7: Inference Pipeline

In Phase 2, performed independently for every incoming analysis request, the submitted text is normalised using the same preprocessing pipeline used at training time (Section 4.5), encoded into integer token identifiers by the already-fitted tokenizer, padded/truncated to 300 tokens, and passed through the loaded model to obtain a raw spam probability. This raw probability, the threshold decision, the calibrated confidence (Section 4.10), the resulting risk band, and the request's processing latency are then returned together as part of the response. Because a single loaded model instance is shared across concurrent requests, the model call itself is made in a thread-safe manner so that multiple simultaneous analysis requests cannot interfere with one another.

The classification threshold is fixed at 0.5, meaning an e-mail is classified as SPAM whenever the model's probability estimate is 0.5 or higher. In addition to this binary decision, the calibrated probability is mapped onto one of three discrete risk bands, shown in Figure 4.8, so that a human reviewer can immediately distinguish a confidently-negative message from one

that only narrowly cleared the classification threshold: a LOW risk band (0.00–0.49) corresponds to predicted NOT SPAM messages, while MEDIUM (0.50–0.79) and HIGH (0.80–1.00) both correspond to predicted SPAM messages but communicate materially different degrees of model confidence, which is useful when prioritising which flagged e-mails a human reviewer should examine first.

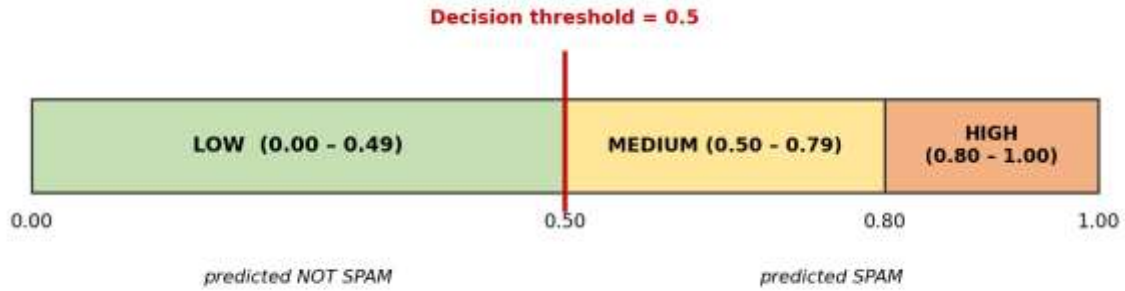


Figure 4.8: Risk Bands mapped onto the calibrated spam probability

4.10 Calibration

A neural classifier's raw sigmoid output is not automatically a well-calibrated probability: a model can be systematically over-confident (or under-confident) even while remaining highly accurate in its thresholded decisions, because accuracy only depends on which side of the threshold a prediction falls on, not on how far from the threshold it is. SignalGuard addresses this using temperature scaling, a simple and widely used post-hoc calibration technique, applied as shown in Figure 4.9:

Raw probability $p \rightarrow \text{Logit } z = \ln(p / (1 - p)) \rightarrow \text{Temperature scaling } z' = z / T \rightarrow \text{Calibrated probability } p' = \sigma(z')$

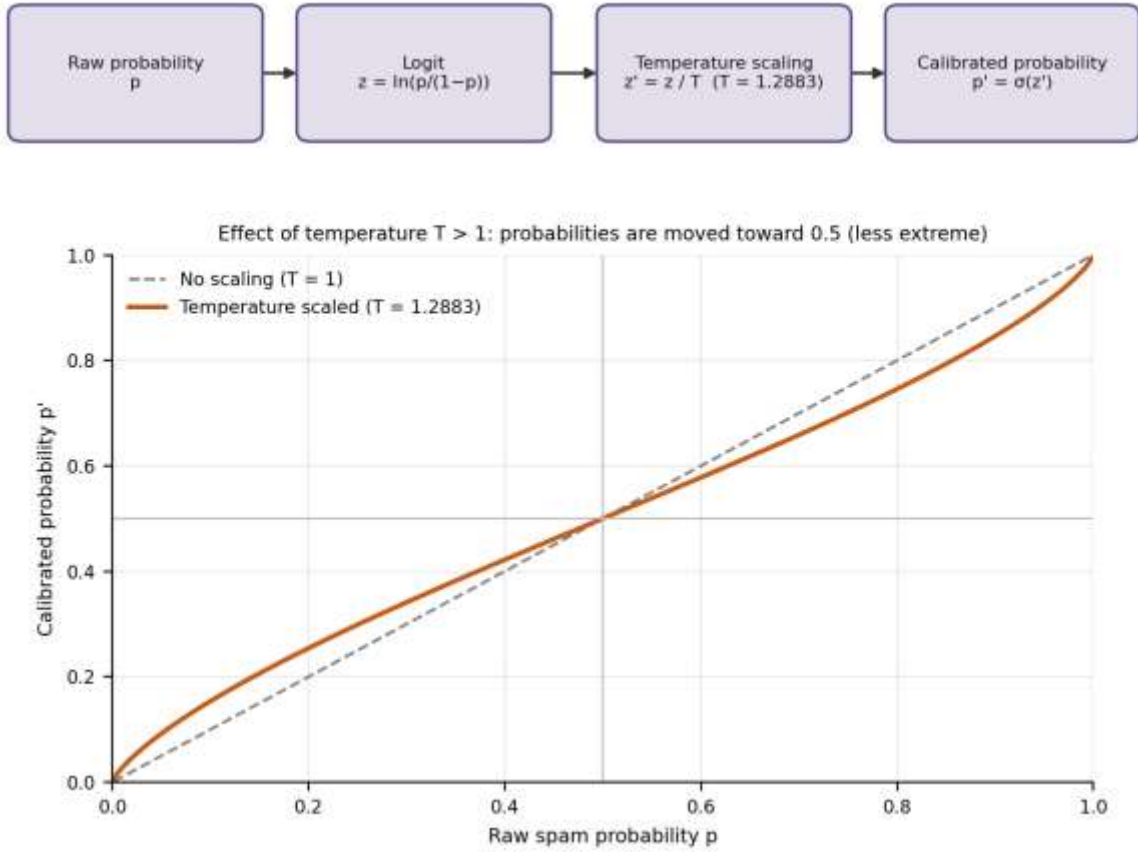


Figure 4.9: Calibration Pipeline (top) and effect of temperature scaling (bottom)

Here $\sigma(\cdot)$ denotes the logistic sigmoid function. A single scalar temperature T is fitted on the validation split (3,415 samples) by finding the value of T that minimises the Negative Log-Likelihood of the calibrated probabilities against the true validation labels, while leaving the model's own weights completely unchanged; because dividing every logit by the same $T > 1$ does not change which class has the higher probability, calibration never changes the model's thresholded SPAM/NOT SPAM decision or its accuracy – it only changes how extreme (close to 0 or 1) the reported probability is allowed to be. The fitted temperature for the current SignalGuard model is $T = 1.2883$; since $T > 1$, this has the effect of pulling the model's raw probabilities slightly back toward 0.5, i.e. making its stated confidence somewhat less extreme, which is the correction that would be expected if the uncalibrated model were, before scaling, slightly over-confident on the validation split.

Calibration quality is measured using two standard metrics: Expected Calibration Error (ECE), which buckets predictions by confidence and measures the average gap between confidence and actual accuracy within each bucket, and Negative Log-Likelihood (NLL), the average of $-\ln(p')$ for the true class, which penalises confident-and-wrong predictions especially heavily. On the

3,415-sample validation split, temperature scaling reduces ECE from 0.00483 to 0.00274 and NLL from 0.02885 to 0.02625 (Chapter 5, Figure 5.4), indicating a modest but consistent improvement in how trustworthy the reported probabilities are as literal probability estimates, on top of an already low starting calibration error.

4.11 Robustness and Security

4.11.1 Robustness Analysis

A trustworthy text classifier should not flip its decision in response to changes that are obviously inconsequential to a human reader. SignalGuard's robustness-analysis module (Figure 4.10) evaluates this directly, rather than assuming it: starting from a baseline e-mail and the model's baseline decision d_0 , the module generates a defined set of transformed variants covering capitalisation changes, punctuation changes, URL-formatting changes, harmless additional spacing, invisible-character insertion, look-alike (homoglyph) character substitution, and word-separator changes. Each transformed variant is passed through the same preprocessing pipeline, tokenizer, and padding step as any other input, and then through the actual trained LSTM model – not a proxy or simplified model – to obtain a decision d_i for that variant.

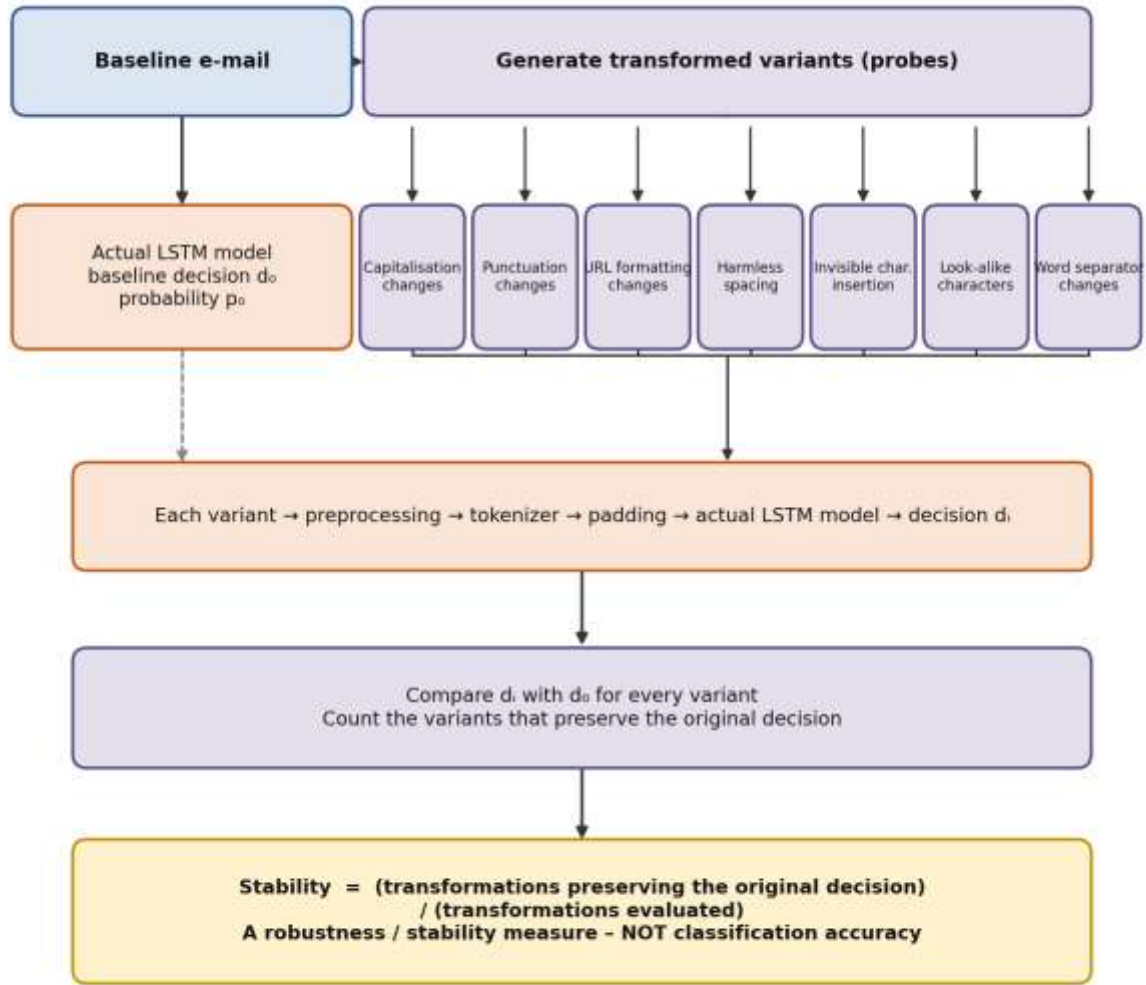


Figure 4.10: Robustness Testing Flow

The resulting stability score is computed as:

$$\text{Stability} = (\text{number of transformations that preserve the original decision}) / (\text{total number of transformations evaluated})$$

It is important to state clearly what this metric is and is not: stability is a measure of decision robustness to the specific set of harmless probe transformations implemented, and is not classification accuracy – a model could in principle have a high stability score while still being systematically wrong, if it is simply consistently wrong in the same way across all of a variant's transformations. Stability and the offline evaluation metrics of Chapter 5 therefore answer different questions and should be read together, not interchangeably.

4.11.2 Unicode Security Analysis

In parallel with, and independently of, robustness probing, SignalGuard performs a Unicode-security analysis of the submitted text, detecting: zero-width characters; bidirectional-control characters (of the kind exploited in the Trojan Source class of attacks discussed in Chapter 2); other Unicode format and control characters that have no legitimate display purpose in e-mail text; unusual space characters (beyond the ordinary ASCII space); control characters; mixed-script tokens (a single token mixing, for example, Latin and Cyrillic letters); and homoglyph patterns, where a character from one script is substituted for a visually similar character from another script.

These characters matter for e-mail security because invisible or reordering Unicode characters can be used to defeat simple keyword-based filters (by breaking up a flagged word with an invisible character that a naive filter's tokeniser does not expect) and to make a link or sender name appear, to the human eye, different from what is actually encoded and processed by downstream systems. By explicitly detecting and reporting these patterns as a distinct finding, rather than silently normalising them away and saying nothing, SignalGuard gives a human reviewer visibility into text-level manipulation attempts that would otherwise be invisible even after the text has been cleaned for classification.

4.11.3 URL and Social-Engineering Signals

SignalGuard also implements a deterministic, rule-based signal layer that looks for patterns commonly associated with phishing and social engineering: urgency language, authority impersonation, fear or threat language, promised rewards, requests for credentials, requests for financial information or action, account-verification requests, and language that attempts to bypass normal organisational procedures. In parallel, the same layer inspects any URLs present in the (unmasked, pre-tokenisation) text for suspicious characteristics: IP-address-based URLs, punycode-encoded domains, unusually deep sub-domain structures, suspicious top-level domains, plain (non-HTTPS) HTTP links, hidden HTML elements, and mismatches between an anchor's visible text and its actual target URL.

It is important to be explicit that these heuristic signals are deterministic, rule-based indicators, not the output of the LSTM neural network, and this report and the underlying API response keep the two clearly separate at every point: the neural network's prediction is reported as the

model's classification, while these heuristics are reported as supporting, human-readable evidence that a reviewer can use alongside, but not instead of, the model's own decision.

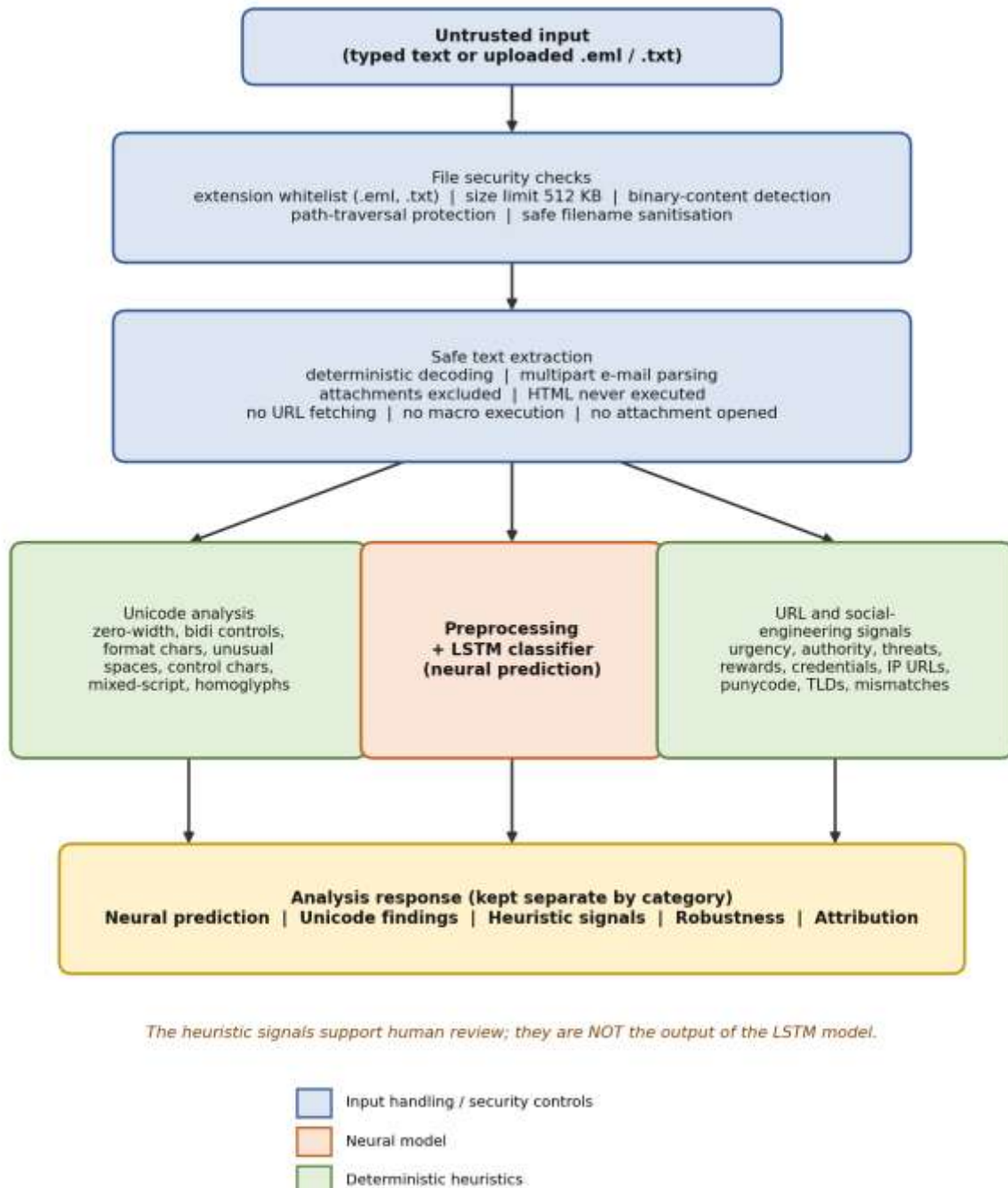


Figure 4.11: Security Analysis Flow – Unicode and URL/social-engineering signals are computed alongside, and reported separately from, the LSTM's neural prediction

4.12 Attribution

To make the LSTM's decision more interpretable without introducing a separate, potentially inconsistent explanation model, SignalGuard implements a position-preserving occlusion-based

attribution method, illustrated in Figure 4.12. Given an already-encoded e-mail, a set of active (non-padding) tokens is selected, subject to an attribution budget that bounds how many tokens are evaluated for a single request. For each selected token in turn, that single token is replaced with the tokenizer's out-of-vocabulary token while the sequence length and every other token's position is left completely unchanged, and the modified sequence is passed through the same trained model used for the main prediction.

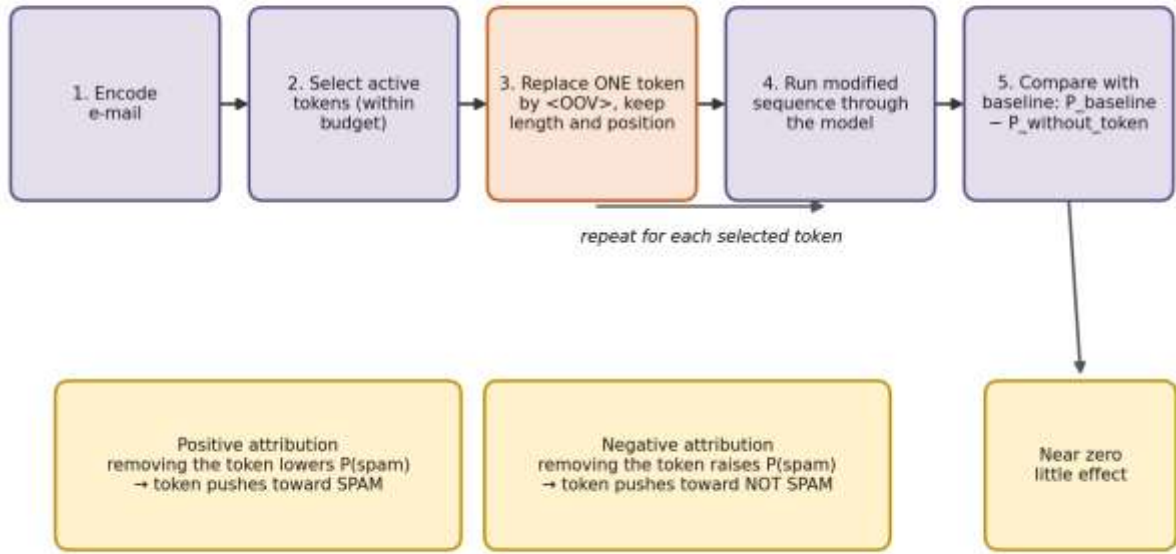


Figure 4.12: Token-Attribution Flow (position-preserving occlusion)

The attribution score for a token is then defined as:

$$\text{Attribution}(\text{token}) = P_{\text{baseline}} - P_{\text{baseline, without token}}$$

A positive attribution score means that removing the token lowered the predicted spam probability, i.e. the token was pushing the model's decision toward SPAM; a negative attribution score means that removing the token raised the predicted spam probability, i.e. the token was pushing the decision toward NOT SPAM; and a near-zero score indicates that the token had little influence on the model's output in the context of that specific e-mail. Because this method only ever queries the model's final output probability, and never inspects internal gradients, activations, or a separately trained surrogate model, it is explicitly not SHAP and not Integrated Gradients – both of which are established but methodologically distinct explainability

techniques – and this report consistently refers to it only as occlusion-based, model-output attribution.

4.13 Web Application

4.13.1 Architecture

SignalGuard is delivered as a Flask web application, illustrated in Figure 4.13, comprising a presentation layer (Jinja2 HTML templates, CSS styling and a JavaScript client that calls the JSON API), an application layer of Flask routes, a service layer of Python modules implementing preprocessing, inference, calibration, Unicode analysis, URL/social-engineering signal extraction, robustness probing and token attribution, and a set of persisted artifacts (the trained model, tokenizer, metadata/calibration/evaluation JSON files, and the analysis history).

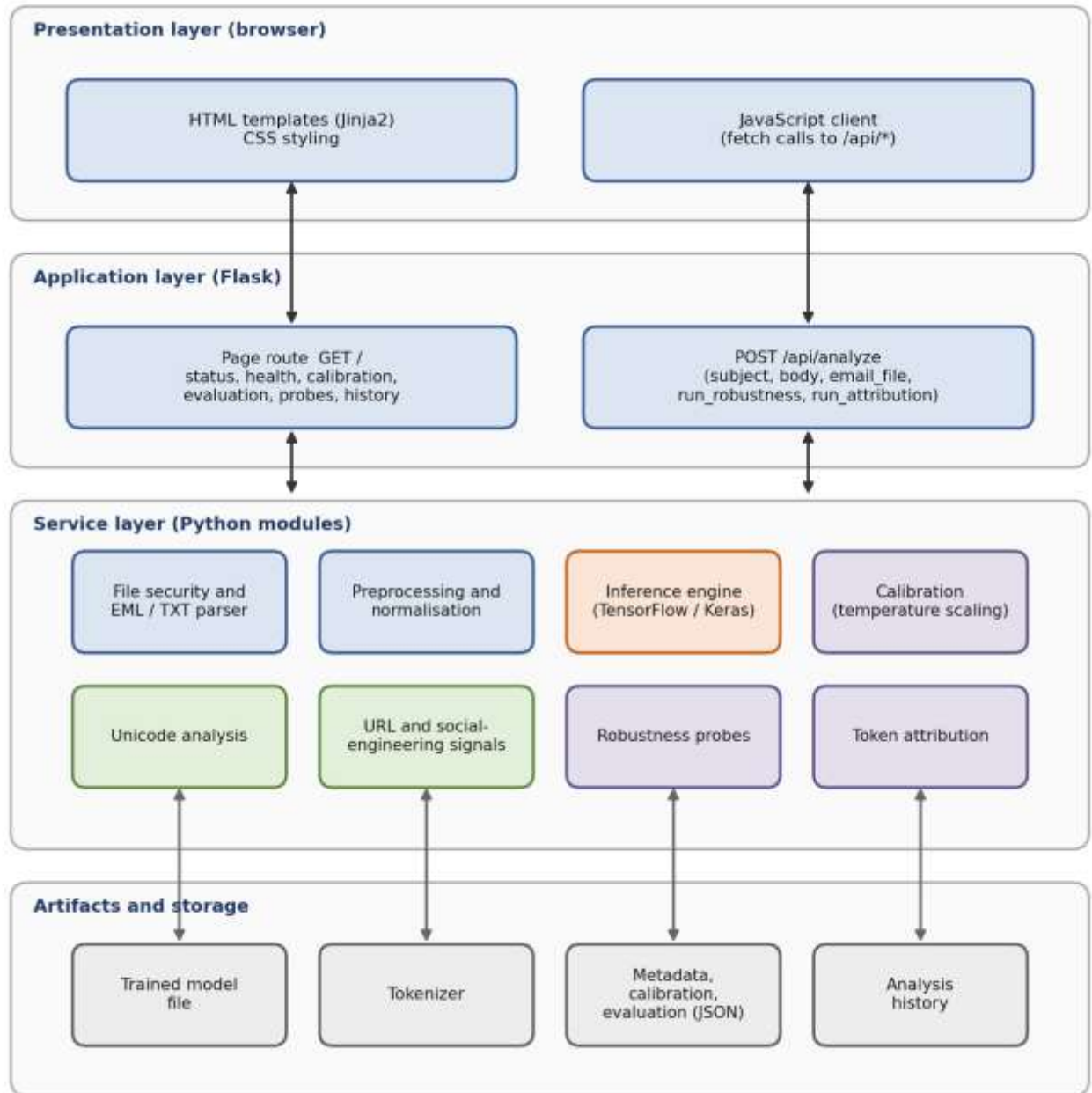


Figure 4.13: Web Application Architecture

4.13.2 API Endpoints

Table 4.3 lists the endpoints exposed by the SignalGuard web application.

Method	Endpoint	Purpose
GET	/	Serves the main web interface.
GET	/api/status	Reports whether the model and supporting artifacts have loaded successfully.

Method	Endpoint	Purpose
POST	/api/analyze	Accepts an e-mail (text or file) and returns the full analysis described below.
GET	/api/probes	Returns the set of robustness probe transformations available.
GET	/api/health	Basic application liveness/health check.
GET	/api/calibration	Returns the fitted temperature and calibration metrics (ECE, NLL).
GET	/api/evaluation	Returns the offline test-set evaluation metrics (Chapter 5).
GET	/api/history	Returns previously recorded analyses.
POST	/api/history/clear	Clears the stored analysis history.

Table 4.3: SignalGuard API endpoints

The POST /api/analyze endpoint is the core of the API. It accepts a subject and body (or an email_file upload), and two optional boolean flags, run_robustness and run_attribution, together with an attribution_budget that bounds the computational cost of the attribution step for that request. Its JSON response is organised into clearly separated sections: model (the LSTM's prediction, probability, and calibrated confidence), content (the normalised text actually seen by the model), normalization (a record of which preprocessing steps altered the input), unicode (Unicode-security findings), signals (deterministic URL/social-engineering findings), robustness (the stability score and per-probe results, if requested), attribution (per-token attribution scores, if requested), model information (architecture and version metadata), timings (a latency breakdown of the request), and privacy (information about what was and was not retained from the request). Structuring the response this way at the API level is what makes it possible, at every layer of the system, to keep the neural network's own prediction visually and structurally separate from the heuristic and diagnostic information that supports it.

4.13.3 Email File Security

SignalGuard accepts uploaded e-mail files in .eml and .txt format, up to a maximum size of 512 KB. Because an uploaded file is, by definition, untrusted input that could originate from an attacker rather than a well-behaved user, the upload path applies a dedicated set of security controls before the file's content is ever passed to preprocessing: extension validation against an explicit whitelist; enforcement of the 512 KB maximum size; binary-content detection, to reject files that are not genuinely plain-text/e-mail content; path-traversal protection and safe filename sanitisation, to prevent a maliciously crafted filename from being used to write or read outside

the intended directory; deterministic decoding (avoiding encoding auto-detection heuristics that could themselves be manipulated); correct multipart MIME parsing for .eml files; and explicit exclusion of any attachments found inside a multipart message, so that only the textual subject/body content is ever passed on for classification.

Beyond these input-validation controls, SignalGuard's e-mail parsing follows a strict non-execution policy: embedded HTML is never rendered or executed, only parsed as text and then stripped, as described in Section 4.5; no URL found in an uploaded e-mail is ever fetched by the server; no macro or script content is ever executed; and no attachment is ever opened, decoded as a document, or otherwise processed beyond being recognised and excluded. This policy exists because an uploaded e-mail must be treated as untrusted input for as long as it is being handled by the server – an attacker who can influence what a spam-detection tool itself does when it opens a malicious e-mail has turned a defensive tool into an attack surface, which SignalGuard's design is intended to avoid entirely.

4.14 Tools and Technologies Used

Table 4.4 lists the technologies used to implement SignalGuard, all of which are actually used by the project as described in the preceding sections.

Technology	Role in SignalGuard
Python	Primary implementation language for preprocessing, modelling, the security/robustness/attribution modules, and the Flask backend.
TensorFlow / Keras	Defines, trains and serves the Embedding + LSTM classification model.
Flask	Serves the web application and the JSON API endpoints listed in Table 4.3.
Jinja2 / HTML / CSS	Server-rendered templates and styling for the web interface.
JavaScript	Client-side logic that calls the /api/* endpoints and renders the analysis results.
NumPy	Numerical operations used throughout preprocessing, calibration, and metric computation.
Pandas	Loading, cleaning and splitting the CEAS_08 dataset during training.
Matplotlib	Generation of the evaluation and calibration charts referenced in this report and used during model analysis.
JSON	Interchange format for the API responses, metadata, calibration and evaluation artifacts.

Technology	Role in SignalGuard
Pytest	Automated unit-testing framework used for the test suite described in Chapter 5.

Table 4.4: Tools and technologies used in SignalGuard

5. RESULTS AND DISCUSSION

5.1 Experimental Environment

All training and evaluation was carried out using the Python-based deep-learning stack described in Table 4.4 (TensorFlow/Keras for modelling, NumPy and Pandas for data handling). Training used the fixed random seed described in Section 4.8 (seed = 42, applied to Python, NumPy and TensorFlow) together with the fixed, stratified 80/10/10 CEAS_08 split described in Section 4.4, so that the results reported in this chapter are reproducible given the same dataset, code and seed.

5.2 Dataset Results

As described in Section 4.4, CEAS_08 comprises 34,155 e-mails, approximately balanced between 17,240 NOT SPAM (50.48%) and 16,915 SPAM (49.52%) examples. The stratified split produced 27,323 training e-mails, 3,415 validation e-mails, and 3,416 test e-mails (Table 4.1, Figure 4.3), with the class ratio preserved in every partition by construction of the stratified split.

5.3 Model Performance

The trained LSTM classifier (Section 4.7), trained according to the methodology in Section 4.8, was evaluated once on the held-out, 3,416-e-mail test split. Table 5.1 and Figure 5.1 summarise the resulting metrics; all figures below refer strictly to this CEAS_08 test split, and, as discussed further in Section 5.11 and Chapter 6, should not be read as a claim of equivalent performance on other, unseen e-mail sources.

Metric	Value
Accuracy	99.33%
Precision	99.41%
Recall	99.23%
F1-Score	99.32%
ROC-AUC	99.89%
PR-AUC	99.89%

Metric	Value
False Positive Rate (FPR)	0.58%
False Negative Rate (FNR)	0.77%

Table 5.1: SignalGuard test-set evaluation metrics (CEAS_08, $n = 3,416$)

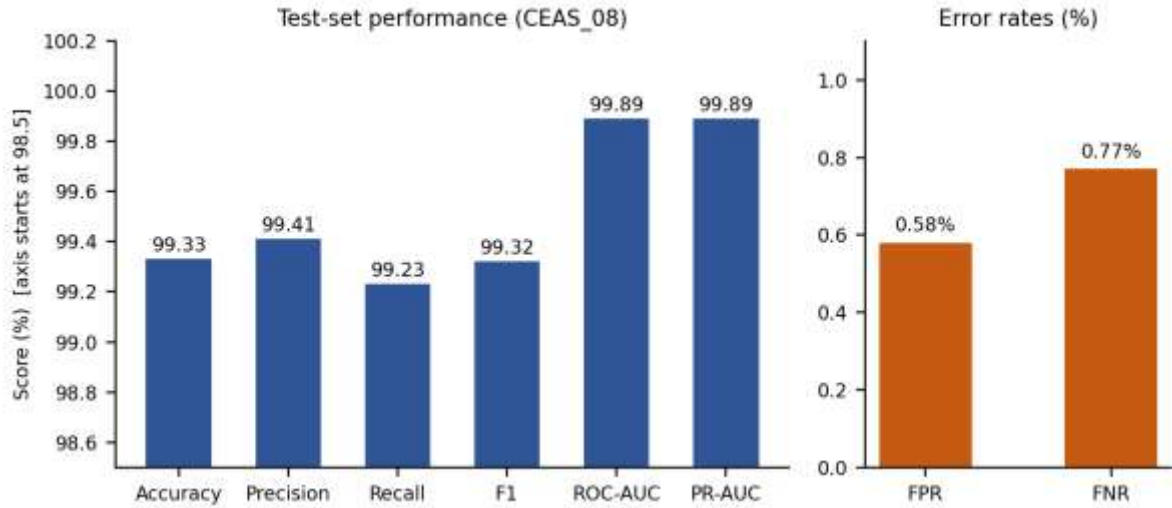


Figure 5.1: Test-set performance metrics (left) and error rates (right)

5.4 Confusion Matrix

Table 5.2 and Figure 5.2 present the confusion matrix computed on the 3,416-e-mail test split.

Actual \ Predicted	NOT SPAM	SPAM
NOT SPAM	1,714 (TN)	10 (FP)
SPAM	13 (FN)	1,679 (TP)

Table 5.2: Confusion matrix on the CEAS_08 test split

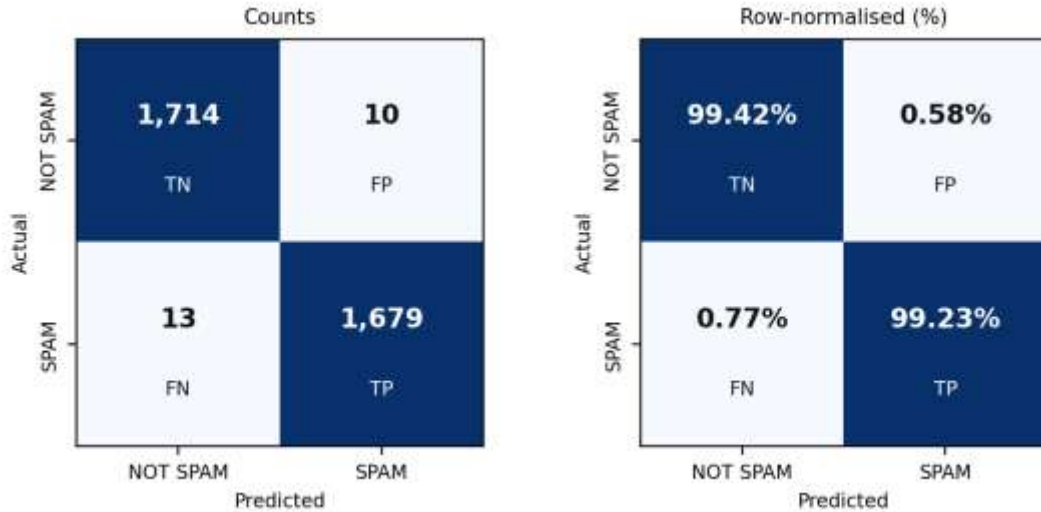


Figure 5.2: Confusion matrix – raw counts (left) and row-normalised percentages (right)

Of the 1,724 genuinely NOT SPAM test e-mails, 1,714 were correctly classified as NOT SPAM (True Negatives) and 10 were incorrectly classified as SPAM (False Positives). Of the 1,692 genuinely SPAM test e-mails, 1,679 were correctly classified as SPAM (True Positives) and 13 were incorrectly classified as NOT SPAM (False Negatives). In the context of a spam filter, a False Positive (a legitimate e-mail wrongly flagged as spam) is generally considered more costly to a user than a False Negative (a spam e-mail that slips through), since a missed legitimate e-mail can have direct real-world consequences; it is therefore notable that, on this test split, the False Positive count (10) is lower than the False Negative count (13), i.e. the model on this dataset is very slightly biased toward under-flagging rather than over-flagging, from an already very small error base of 23 mistakes out of 3,416 e-mails.

5.5 Evaluation Metrics: Formulas and Interpretation

Using $TP = 1,679$, $TN = 1,714$, $FP = 10$ and $FN = 13$, the metrics in Table 5.1 are computed as follows.

Accuracy

$$Accuracy = (TP + TN) / (TP + TN + FP + FN) = (1,679 + 1,714) / 3,416 = 99.33\%$$

Accuracy is the overall proportion of e-mails correctly classified, irrespective of class; because CEAS_08 is close to balanced, accuracy is a reasonably meaningful headline metric here, though it is still read alongside precision, recall and the confusion matrix rather than in isolation.

Precision

$$\text{Precision} = TP / (TP + FP) = 1,679 / (1,679 + 10) = 99.41\%$$

Precision measures, of the e-mails the model flagged as SPAM, what proportion were genuinely spam; a high precision (99.41%) indicates that when SignalGuard flags an e-mail as spam, that flag is very rarely a false alarm on this test split.

Recall

$$\text{Recall} = TP / (TP + FN) = 1,679 / (1,679 + 13) = 99.23\%$$

Recall (also called sensitivity or the true-positive rate) measures, of the e-mails that were genuinely spam, what proportion the model successfully caught; a recall of 99.23% indicates that only a small fraction of test-set spam e-mails passed through undetected.

F1-Score

$$F1 = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall}) = 99.32\%$$

The F1-score is the harmonic mean of precision and recall, and is useful as a single summary number when both false positives and false negatives matter; SignalGuard's F1-score of 99.32% indicates a good balance between the two on this test split.

False Positive Rate (FPR)

$$FPR = FP / (FP + TN) = 10 / (10 + 1,714) = 0.58\%$$

FPR measures the proportion of genuinely legitimate e-mails that were incorrectly flagged as spam; a low FPR is particularly important for a spam filter, since false positives risk hiding important legitimate communication from the user.

False Negative Rate (FNR)

$$FNR = FN / (FN + TP) = 13 / (13 + 1,679) = 0.77\%$$

FNR measures the proportion of genuinely spam e-mails that the model missed; SignalGuard's FNR of 0.77% indicates a low miss rate on this test split, though, as with every metric in this section, this figure characterises performance specifically on CEAS_08-style spam.

ROC-AUC (99.89%) and PR-AUC (99.89%) summarise the model's discriminative ability across all possible classification thresholds, rather than only at the fixed 0.5 threshold used for

the reported decisions; values this close to the maximum of 100% indicate that the model's ranking of e-mails by predicted spam probability is very consistent with the true labels on this test split, independent of exactly where the 0.5 cut-off is placed.

5.6 Calibration Results

Table 5.3 and Figure 5.3 report the effect of the temperature-scaling calibration described in Section 4.10, measured on the 3,415-sample validation split, using the fitted temperature $T = 1.2883$.

Metric	Before scaling	After scaling	Change
Expected Calibration Error (ECE)	0.00483	0.00274	−44.3%
Negative Log-Likelihood (NLL)	0.02885	0.02625	−9.0%

Table 5.3: Effect of temperature scaling on validation-set calibration ($n = 3,415$, $T = 1.2883$)

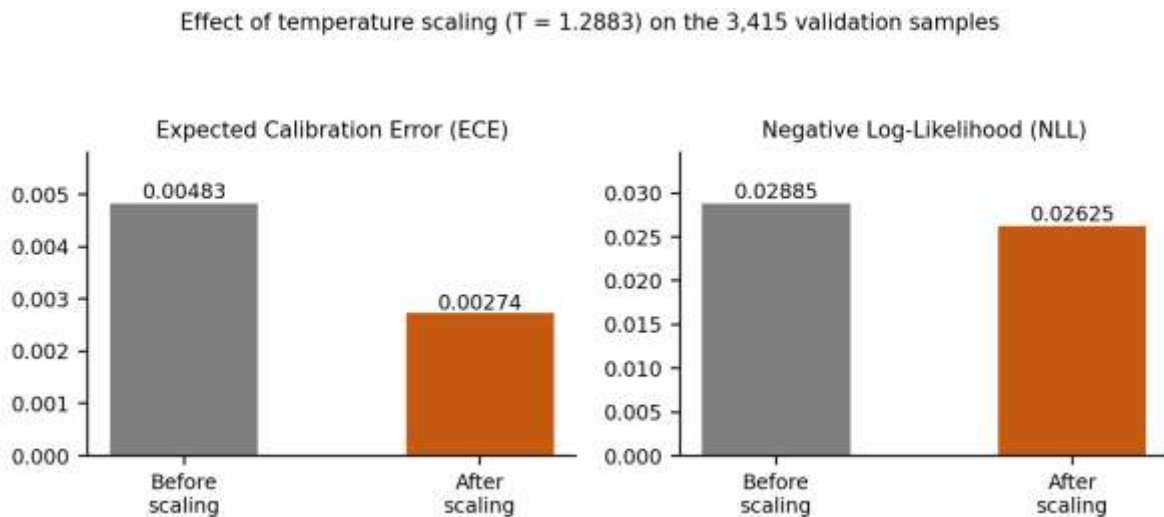


Figure 5.3: ECE and NLL before and after temperature scaling

Both ECE and NLL decrease after temperature scaling, confirming that the reported confidence values are somewhat more trustworthy as literal probability estimates after calibration than the model's raw, uncalibrated sigmoid outputs. It should be noted that the pre-calibration ECE (0.00483) was already quite low in absolute terms, so the practical benefit of calibration here is a further refinement of an already reasonably well-calibrated model, rather than a correction of severe miscalibration; as explained in Section 4.10, calibration never changes the model's underlying SPAM/NOT SPAM decision or the metrics reported in Section 5.3, since it is a monotonic rescaling of the same underlying logit.

5.7 Robustness Analysis

The robustness-probing module described in Section 4.11.1 evaluates the stability of the model's decision under a defined set of harmless transformations: capitalisation changes, punctuation changes, URL-formatting changes, harmless spacing, invisible-character insertion, look-alike (homoglyph) character substitution, and word-separator changes. For a given e-mail, the module reports the fraction of these transformations whose model decision agrees with the untransformed baseline decision, as defined by the stability formula in Section 4.11.1.

Because this stability score is computed per analysed e-mail rather than as a single fixed offline benchmark figure, this report does not state a single project-wide stability percentage; instead, the mechanism itself – re-running the actual trained model on every transformed variant and directly comparing decisions – is the result being reported here, consistent with the instruction to avoid inventing results that were not part of the documented project. As emphasised in Section 4.11.1, this stability score characterises decision robustness to the implemented probe transformations and is a distinct concept from the classification-accuracy metrics of Section 5.3; a complete robustness evaluation summary (aggregated stability across a representative e-mail sample) is identified as a natural extension of this analysis in the Future Scope (Section 6.2).

5.8 Testing Results

The SignalGuard codebase includes an automated Pytest-based test suite comprising 8 passing tests, covering: text preprocessing; URL and e-mail-address normalisation; label mapping; .eml parsing; handling of invalid uploads; metric-bound sanity checks (that reported metrics fall within valid ranges); probability-threshold behaviour (that the 0.5 threshold is applied correctly); risk-band behaviour (that probabilities map to the correct LOW/MEDIUM/HIGH band); and hidden-Unicode removal. Table 5.4 summarises this test coverage.

#	Test Area	What is verified
1	Text preprocessing	The cleaning pipeline (Section 4.5) transforms sample inputs as expected.
2	URL/e-mail normalisation	URLs and e-mail addresses are correctly replaced with URLTOKEN / EMAILTOKEN.
3	Label mapping	Supported dataset label formats map correctly to the {0,1} training labels.
4	EML parsing	Multipart .eml files are parsed correctly and attachments are excluded.
5	Invalid uploads	Files failing extension, size or binary-content checks are correctly rejected.

#	Test Area	What is verified
6	Metric bounds	Reported evaluation metrics fall within valid (0–100% / 0–1) ranges.
7	Threshold behaviour	The 0.5 classification threshold is applied correctly to the spam probability.
8	Risk-band behaviour	Calibrated probabilities map to the correct LOW / MEDIUM / HIGH risk band.

Table 5.4: Automated test coverage (8 passing tests)

All 8 tests pass at the time of writing. No additional test results beyond these 8 are claimed; broadening automated test coverage (for example, to the robustness and attribution modules) is identified as future work in Section 6.2.

5.9 Functional Requirements Results Achieved

Table 5.5 maps each functional requirement from Table 3.1 to the component of SignalGuard that satisfies it and the evidence presented in this report.

ID	Status	Evidence
FR-1	Met	Direct subject/body text input, Section 4.13.2 (POST /api/analyze).
FR-2	Met	eml/.txt upload with 512 KB limit, Section 4.13.3.
FR-3	Met	LSTM classification, Sections 4.7, 5.3, 5.4.
FR-4	Met	Unicode-security analysis, Section 4.11.2.
FR-5	Met	Robustness probing and stability score, Sections 4.11.1, 5.7.
FR-6	Met	Token attribution, Section 4.12.
FR-7	Met	/api/history and /api/history/clear endpoints, Section 4.13.2.
FR-8	Met	Documented JSON API, Section 4.13.2, Table 4.3.

Table 5.5: Functional requirements results achieved

5.10 Non-Functional Requirements Results Achieved

ID	Status	Evidence / Notes
NFR-1	Partially evaluated	Per-request timing breakdown is returned by the API (Section 4.13.2); no formal load/throughput testing has been performed.
NFR-2	Met	File-security controls of Section 4.13.3 (extension, size, binary detection, path-traversal protection).

ID	Status	Evidence / Notes
NFR-3	Met	Artifacts loaded once at start-up and re-used; thread-safe model access (Section 4.9).
NFR-4	Met	Organised web interface presenting prediction, probability, risk band and supporting analyses (Section 4.13.1).
NFR-5	Met	Modular separation of preprocessing, modelling, calibration, security, robustness, and attribution (Sections 4.5–4.12).
NFR-6	Met	Robustness stability score computed per analysis, Section 4.11.1 / 5.7.

Table 5.6: Non-functional requirements results achieved

5.11 Comparison with Existing Approaches

Table 5.7 places SignalGuard's CEAS_08 test-set results alongside same-dataset figures reported in the literature surveyed in Chapter 2. These comparisons are same-dataset-to-same-dataset where possible, and the table deliberately also reproduces the cross-dataset generalisation caveat reported by that same literature, since it is directly relevant to how SignalGuard's own figures should be interpreted.

Study	Method	Reported same-dataset result
SignalGuard (this work)	Embedding + LSTM	99.33% accuracy, F1 = 99.32% on CEAS_08 test split
Rabbi, Champa & Zibran (2023)	ML/NLP pipeline on phishing e-mail text	High accuracy, precision and recall reported on two public phishing datasets
Al-Subaiey et al. (2024)	ML model on a large public phishing dataset, with XAI	F1-score of 0.99
Pajola et al. (2025)	TF-IDF (LR/NB/RF/SVM/MLP) and fine-tuned DistilBERT, evaluated on CEAS and 7 other datasets	Near-perfect same-dataset F1 (e.g. 0.98–1.00 on CEAS), but F1 drops substantially – in some cases toward 0 – when the same model is tested on a different dataset

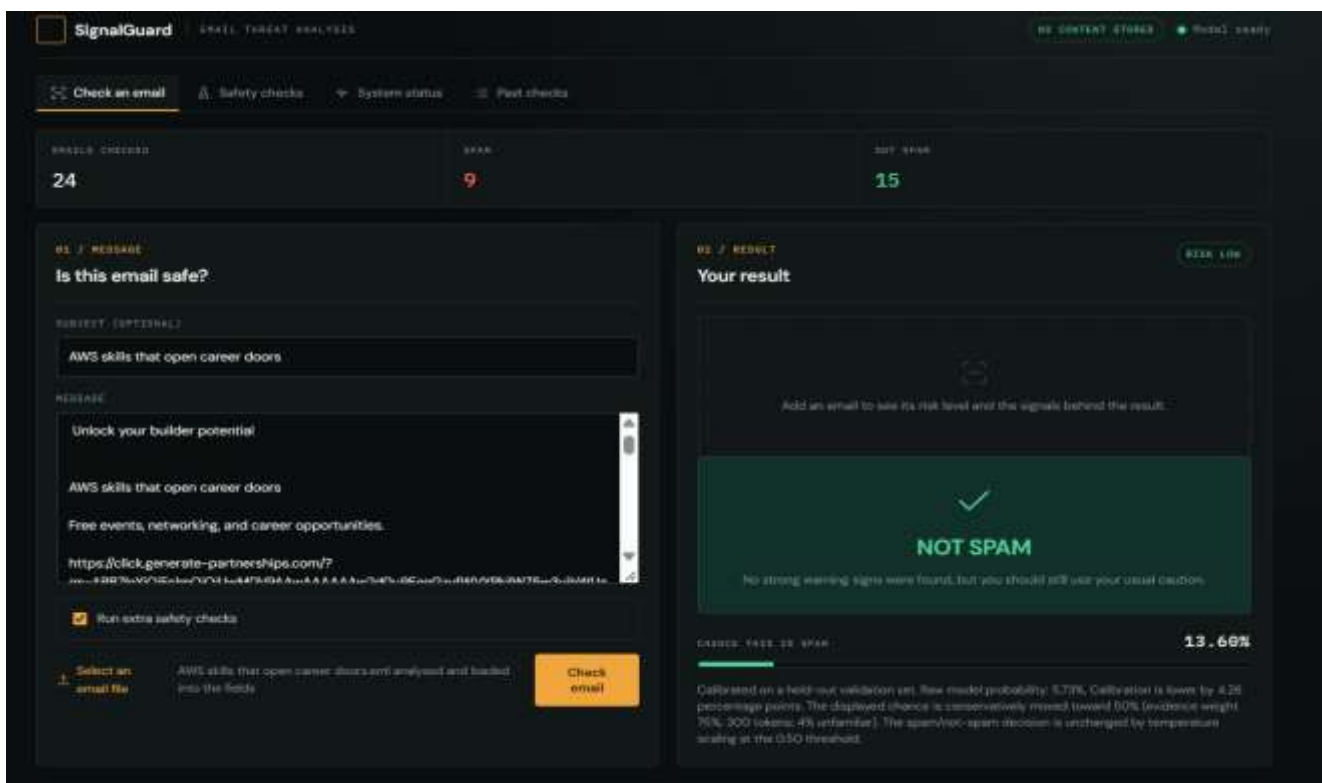
Table 5.7: Comparison with existing approaches reported in the literature (Chapter 2)

Two observations follow from this comparison. First, SignalGuard's same-dataset test performance is consistent with the general order of magnitude reported by other LSTM- and ML-based e-mail classifiers evaluated on a single dataset, which supports the correctness of the modelling approach described in Chapter 4. Second, and more importantly, the cross-dataset

results reported by Pajola et al. (2025) and the generalisation-failure findings of Champa, Rabbi and Zibran (2024b) (Chapter 2) show that near-perfect same-dataset performance of the kind reported in Table 5.7 is not, by itself, evidence that a model will perform similarly well on a different e-mail source or on more recent phishing campaigns. Because SignalGuard has been trained and evaluated only on CEAS_08, no direct cross-dataset comparison is available for SignalGuard itself at this time; this is recorded here as an explicit limitation (Section 5.14 note and Chapter 6) rather than a claim that is not supported by an actual experiment.

5.12 Project GUI / System Screenshots

The SignalGuard web interface (Section 4.13) presents an input form for the e-mail subject and body (or a file-upload control for .eml/.txt files), and, after submission, an organised results view showing the SPAM/NOT SPAM decision, the calibrated probability and risk band, the Unicode-security findings, the URL/social-engineering signals, and, when requested, the robustness stability score and the token-attribution results. Live screenshots of the running application are intended to be captured from the deployed instance and inserted here (Figures 5.4a–5.4d) as part of final report preparation; they have intentionally been left out of this automatically generated draft rather than being fabricated, consistent with the instruction not to invent screenshots that were not actually captured from the running system.



SignalGuard

EMAIL THREAT ANALYSIS

NO CONTENT STORED

Model ready

Check an email

Safety checks

System status

Past checks

EMAIL CHECKS

23

SPAM

9

NOT SPAM

14

01 / MESSAGE

Is this email safe?

SUBJECT (OPTIONAL)

URGENT: Your PayPal Account Has Been Suspended

MESSAGE

Dear Customer,

URGENT ACTION REQUIRED - Your account will be suspended within 24 hours!

We have detected suspicious login activity on your PayPal account. Due to unusual activity, your account has been temporarily locked and will be permanently suspended unless you verify your identity immediately.

Please click here to verify your account.

☒ Run extra safety checks

Select an email file

phishing email analysed and loaded into the fields

Check email

02 / RESULT

YOUR RESULT

ADD MESSAGE

SPAM

Be careful with this message. Check the sender before responding or opening links.

CHANCE THIS IS SPAM

87.32%

Calculated on a held-out validation set. Raw model probability: 96.22%. Calibration is higher by 154 percentage points. The displayed chance is conservatively moved toward 50% (evidence weight 75%, 100 tokens, 0% unfamiliar). The spam/not-spam decision is unchanged by temperature scaling at the 0.50 threshold.

SignalGuard

EMAIL THREAT ANALYSIS

NO CONTENT STORED

Model ready

Check an email

Safety checks

System status

Past checks

SAFETY CHECKS

Prediction stability analysis

RESULTS

STARTING RESULT

SPAM

SAME ANSWER EACH TIME

95.0%

ANSWERS THAT CHANGED

1

BIGGEST EFFECT

Extra harmless words

CHECKS WITH THE SAME ANSWER

23 of 24

STOPPED BY NORMALIZATION

12 of 24

SAME MODEL THRESHOLD

5 of 24

REACHES A DIFFERENT MODEL INPUT

7 of 24

PERICTION APPLIED TO THIS EMAIL

PIPELINE RESULT

BASILINE CHANCE

TESTED CHANGE

PERCENT

PREDICT CHANCEBEST

Extra harmless words

new model input

87.3%

13.1%

-74.2 pp

YES

Link brackets changed

new model input

87.3%

85.9%

-1.4 pp

NO

Letters changed to numbers

new model input

87.3%

86.7%

-1.6 pp

NO

Look-alike letters

new model input

87.3%

86.7%

-0.6 pp

NO

Punctuation removed

new model input

87.3%

87.1%

-0.3 pp

NO

SignalGuard

EMAIL THREAT ANALYSED

NO CONTENT STORED

Model ready

Check an email

Safety checks

System status

Past checks

LIVE

Model health

MEASURED NOW

ARTIFACT SIZE	SEQUENCE LENGTH	VOCABULARY SIZE	ARCHITECTURE	TRAINING CORPUS	FIRST-CALL TRAINING (PASS AT LOAD)	STINGLE REQUEST, END TO END
22.58 MB	388	28,888	Embedding(96) -> SpatialDropout1D -> LSTM(64) -> Dense(32) -> Dense(1, sigmoid)	CEAS_88.csv	1898.5 ms	38.7 ms
BATCHES PER STEP	BATCH SPEEDUP					
7.3 ms	5.28x					

MOST RECENT EMAIL (METADATA ONLY)

LAST EMAIL RESULT	LAST EMAIL EXPENSE	LAST EMAIL TOTAL REQUEST	LAST EMAIL MODEL TOKENS	LAST EMAIL UNFAMILIAR TOKENS	LAST EMAIL SAFETY STABILITY
SPAM	43.3 ms	388.3 ms	168	0.6%	not requested

ARTIFACTS

Artifact inventory

8 / 8 PRESENT

ARTIFACT TYPE	FILE	PRESENT	SIZE
calibration	calibration.json	present	8.8 KB
evaluation	evaluation.json	present	8.4 KB

SignalGuard

EMAIL THREAT ANALYSED

NO CONTENT STORED

Model ready

Check an email

Safety checks

System status

Past checks

PRIVATE

Session history

Clear

ANALYSED	SPAM	NOT SPAM
23	9	14

CREATED AT	RESULT
8/23/2026, 2:30:30 AM	SPAM
8/23/2026, 2:30:08 AM	NOT SPAM
8/23/2026, 2:29:02 AM	NOT SPAM
8/23/2026, 2:28:58 AM	NOT SPAM
8/23/2026, 2:28:28 AM	NOT SPAM
8/23/2026, 2:26:47 AM	SPAM
8/23/2026, 2:26:38 AM	NOT SPAM
8/23/2026, 2:26:30 AM	SPAM
8/23/2026, 2:24:59 AM	SPAM
8/23/2026, 2:18:35 AM	NOT SPAM
8/23/2026, 2:18:03 AM	NOT SPAM
8/23/2026, 2:17:48 AM	NOT SPAM
8/23/2026, 2:15:23 AM	NOT SPAM

5.13 Societal Impact of the Project

Spam and phishing e-mails impose a measurable social and economic cost through direct financial fraud, credential theft, malware delivery, and the cumulative time individuals and organisations spend triaging unwanted messages. By combining a deep-learning classifier with transparent, separately reported calibration, robustness and explainability information, SignalGuard is intended to support rather than replace human judgement: a security-conscious user or a small-organisation administrator can use the tool to prioritise which e-mails need attention, while the risk-band and attribution information make the tool's reasoning more inspectable than an opaque, purely black-box filter. The project also has direct educational value as a mini-project, since it exposes students to the complete lifecycle of a real deep-learning system, data preparation, model training, calibration, security conscious engineering, and evaluation rather than only the model training step in isolation.

At the same time, the limitations discussed in Section 5.14 and Chapter 6 have a direct societal dimension: a spam filter that is deployed based only on its CEAS_08 performance, without further evaluation on the organisation's own, current e-mail traffic, risks both under-protecting users against modern phishing techniques not represented in the training data, and over-flagging legitimate e-mail in ways that were not measured during this evaluation. Communicating these limitations honestly, as this report attempts to do throughout, is itself part of responsible deployment of a machine-learning-based security tool.

5.14 SDG Mapping

Table 5.8 maps SignalGuard to the United Nations Sustainable Development Goals (SDGs) to which it most plausibly contributes.

SDG	Relevance to SignalGuard
SDG 9: Industry, Innovation and Infrastructure	Applies deep learning and secure software-engineering practice to strengthen the resilience of e-mail communication infrastructure against spam and phishing.
SDG 16: Peace, Justice and Strong Institutions	Contributes to reducing online fraud and cybercrime facilitated by phishing e-mails, and supports safer digital communication for individuals and institutions.
SDG 4: Quality Education	Serves as an educational mini-project demonstrating an end-to-end, security-conscious deep-learning system to undergraduate students.

Table 5.8: SDG mapping for the SignalGuard project

As with the societal-impact discussion above, this mapping reflects the plausible direction of the project's contribution and is not accompanied by a quantitative impact study; it is offered as a qualitative framing exercise appropriate to a mini-project report.

6. CONCLUSION AND FUTURE SCOPE

6.1 Project Strengths

Reflecting on the design and implementation described in Chapter 4 and the results presented in Chapter 5, SignalGuard has the following documented strengths:

- A genuine deep-learning architecture (Embedding + SpatialDropout1D + LSTM + Dense + Sigmoid) rather than a purely rule-based or bag-of-words classifier.
- A tokenizer fitted only on the training split, avoiding vocabulary leakage from validation or test data.
- A stratified data split that preserves class balance across training, validation and test partitions.
- Class-weight support during training to guard against class-imbalance effects.
- A correctly matched sigmoid-output/binary-cross-entropy-loss configuration for binary classification.
- A dedicated URL, e-mail-address and phone-number normalisation step in preprocessing.
- Explicit Unicode-security handling (zero-width characters, bidirectional controls, homoglyphs, and related patterns).
- Defensive, security-conscious handling of untrusted uploaded e-mail files.
- An explicit robustness-testing mechanism that queries the actual trained model rather than assuming stability.

6.2 Project Limitations and Future Scope

The most important limitation of SignalGuard, stated plainly, is that the shipped model is trained and evaluated primarily on the CEAS_08 dataset. As discussed in Chapter 2 (in particular the cross-dataset findings of Pajola et al., 2025, and Champa, Rabbi and Zibran, 2024b) and revisited in Section 5.11, near-perfect performance on a single benchmark dataset is a well-documented and recurring pattern in the phishing/spam-detection literature, and such performance does not, by itself, demonstrate equivalent effectiveness against modern spam, new and evolving phishing campaigns, corporate or organisation-specific e-mail, multilingual e-mail, short promotional messages, cloud-service-impersonation scams, social-media-originated scams, or unusually formatted e-mail – none of which are well represented in the CEAS_08 corpus, which predates many current phishing techniques. This is an instance of domain shift:

the statistical distribution of e-mails the model was trained and tested on may differ, sometimes substantially, from the distribution of e-mails it would encounter in real-world deployment, and the gap between strong benchmark performance and reliable real-world deployment is a recognised open problem in this research area rather than one specific to this project.

Other limitations, tied directly to the design decisions documented in Chapter 4, include: the absence of early stopping and L2 regularisation in the current training procedure (Section 4.8), meaning the model is trained for a fixed number of epochs rather than being automatically stopped or additionally regularised based on validation performance; the model's reliance on a single language (English-language e-mail text, as represented in CEAS_08); the absence of any evaluation of throughput or latency under concurrent load (Table 5.6, NFR-1); and the fact that the robustness-stability metric (Section 4.11.1) is computed per individual analysis rather than as an aggregated, dataset-wide robustness benchmark (Section 5.7).

Building on these documented limitations, the following directions are identified as realistic future work, clearly distinguished here from the functionality that already exists in the current implementation:

- Training and evaluating the model on multiple, more recent and more diverse public e-mail datasets – including the other datasets already available to the project (Section 4.4) – rather than CEAS_08 alone, together with formal cross-dataset evaluation of the kind described in Chapter 2.
- Incorporating modern, real-world e-mail collections that better reflect current phishing and spam tactics, including LLM-generated phishing content.
- Extending the system to multilingual spam and phishing detection, beyond the English-only CEAS_08 corpus.
- Exploring transformer-based architectures (e.g. fine-tuned encoder models) as an alternative or complement to the current LSTM classifier, informed by the comparative findings discussed in Chapter 2.
- Extending the explainability component with additional, complementary methods (for example, gradient-based attribution) alongside the current occlusion-based method, with a clear comparison of what each method does and does not capture.
- Further improving probability calibration, for example by evaluating class-wise or adaptive calibration in addition to the single global temperature currently used.
- Broadening the security analysis to cover additional attack surfaces, such as attachment-content inspection performed in a fully sandboxed manner, while preserving the current policy of never executing untrusted content.

In summary, SignalGuard demonstrates a complete, security-conscious pipeline for deep-learning-based e-mail spam classification, combining a well-established Embedding + LSTM architecture with calibration, Unicode-security analysis, deterministic social-engineering signal extraction, robustness probing, and model-output-based attribution, delivered through a documented web application and API. Its reported test-set metrics on CEAS_08 are strong and consistent with comparable systems in the literature, but this report has deliberately and repeatedly emphasised that such metrics characterise performance on a specific, ageing benchmark dataset rather than guaranteeing equivalent real-world effectiveness; addressing this domain-shift limitation through broader, continuously updated evaluation is the single most important direction for future work on this project.

7. REFERENCES

The following references were consulted in the preparation of this report. Sources directly informing the literature survey of Chapter 2 are marked implicitly by in-text citation; general software/library and dataset references are included for completeness of the tools and dataset descriptions in Chapter 4.

- [1] Al-Subaiey, A., Al-Thani, M., Alam, N. A., Antora, K. F., Khandakar, A., & Zaman, S. A. U. (2024). Novel interpretable and robust web-based AI platform for phishing email detection. *Computers and Electrical Engineering*. Preprint: arXiv:2405.11619.
- [2] Boucher, N., Shumailov, I., Anderson, R., & Papernot, N. (2022). Bad Characters: Imperceptible NLP Attacks. In *Proceedings of the 43rd IEEE Symposium on Security and Privacy (SP)*, pp. 1987–2004. IEEE.
- [3] Boucher, N., & Anderson, R. (2023). Trojan Source: Invisible Vulnerabilities. In *Proceedings of the 32nd USENIX Security Symposium, Anaheim, CA*, pp. 6507–6524. USENIX Association.
- [4] Champa, A. I., Rabbi, M. F., & Zibran, M. F. (2024). Curated Datasets and Feature Analysis for Phishing Email Detection with Machine Learning. In *Proceedings of the 3rd IEEE International Conference on Computing and Machine Intelligence (ICMI)*, pp. 1–7.
- [5] Champa, A. I., Rabbi, M. F., & Zibran, M. F. (2024b). Why Phishing Emails Escape Detection: A Closer Look at the Failure Points. In *Proceedings of the 12th International Symposium on Digital Forensics and Security (ISDFS)*, pp. 1–6. IEEE.
- [6] Dada, E. G., Bassi, J. S., Chiroma, H., Abdulhamid, S. M., Adetunmbi, A. O., & Ajibuwa, O. E. (2019). Machine learning for email spam filtering: review, approaches and open research problems. *Heliyon*, 5(6), e01802.
- [7] Desai, S., & Durrett, G. (2020). Calibration of Pre-trained Transformers. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 295–302. Association for Computational Linguistics.
- [8] Li, J., Monroe, W., & Jurafsky, D. (2017). Understanding Neural Networks through Representation Erasure. arXiv preprint arXiv:1612.08220.

- [9] Pajola, L., Caripoti, E., Banzer, S., Pizzi, S., Conti, M., & Apruzzese, G. (2025). E-PhishGen: Unlocking Novel Research in Phishing Email Detection. In Proceedings of the 2025 Workshop on Artificial Intelligence and Security (AISec), Taipei, Taiwan. ACM.
- [10] Rabbi, M. F., Champa, A. I., & Zibran, M. F. (2023). Phishy? Detecting Phishing Emails Using Machine Learning and Natural Language Processing. In Machine Learning and NLP Applications in Cybersecurity, Springer, pp. 119–134.
- [11] Chollet, F., et al. (2015). Keras. <https://keras.io>.
- [12] Abadi, M., et al. (2016). TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems. arXiv preprint arXiv:1603.04467.
- [13] Pallets Projects. (2010–present). Flask: A lightweight WSGI web application framework for Python. <https://flask.palletsprojects.com>.
- [14] Richardson, L. (2004–present). Beautiful Soup Documentation. <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>.
- [15] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- [16] The pandas development team. (2020). pandas-dev/pandas: Pandas. Zenodo.
- [17] The Unicode Consortium. (2023). The Unicode Standard, Version 15.0 – Unicode Bidirectional Algorithm (Unicode Technical Report #9).
- [18] CEAS 2008 Live Spam Challenge Corpus (CEAS_08). Available at: <https://plg.uwaterloo.ca/~gvcormac/ceascorpus/> (public e-mail spam-classification benchmark used for model training and evaluation, Chapter 4).
- [19] Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks. In Advances in Neural Information Processing Systems (NeurIPS), 25. [Representative deep-learning background reference for neural-network training practice.]
- [20] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.